

Program Studi Matematika
Fakultas Sains
Institut Teknologi Sumatera

# Modul Praktikum
# Pemodelan Komputasi dan Analisis Numerik

MA25-31017

Disusun oleh

Rifky Fauzi
Dear Michiko Mutiara Noor

Lampung Selatan, 2026

# Daftar Isi

Daftar Isi 2

Identitas Modul 7

Petunjuk Umum Praktikum 10

1 MATLAB Dasar 12
- 1.1 Variabel dan operasi dasar . . . 12
- 1.2 Percabangan `if-else` . . . 12
- 1.3 Pengulangan . . . 13
- 1.4 Membuat fungsi . . . 13
- 1.5 Menggambar grafik sederhana . . . 13
- 1.6 Pola kerja driver dan function . . . 14
- 1.7 Galat: definisi sebelum latihan . . . 14
- 1.8 Latihan MATLAB Dasar . . . 15
- 1.9 Latihan fondasi dari Lembar Kerja . . . 15

2 Praktikum 1: Akar Persamaan Nonlinear 17
- 2.1 Masalah pencarian akar . . . 17
- 2.2 Biseksi . . . 18
- 2.3 Regula Falsi . . . 18
- 2.4 Titik Tetap . . . 19
- 2.5 Newton-Raphson . . . 19
- 2.6 Secant . . . 20
- 2.7 Contoh kelas: Biseksi dan Regula Falsi . . . 20
- 2.8 Program MATLAB yang dibahas di kelas: function dan driver . . . 20
- 2.9 Latihan / tugas utama . . . 22
- 2.10 Latihan terintegrasi dari Lembar Kerja . . . 23

3 Praktikum 2: Sistem Persamaan Linear Iteratif 24
3.1 Mulai dari persamaan, bukan matriks . . . 24
3.2 Formula Jacobi . . . 25
3.3 Formula Gauss-Seidel . . . 25
3.4 Baru kemudian notasi matriks . . . 26
3.5 Galat iterasi dan residual . . . 26
3.6 Contoh . . . 26
3.7 Program MATLAB: function dan driver . . . 27
3.8 Grafik konvergensi . . . 28
3.9 Latihan . . . 28
3.10 Latihan terintegrasi dari Lembar Kerja . . . 29

4 Praktikum 3: SPNL dan Pencocokan Kurva 30
4.1 Bagian A — Newton untuk Sistem Persamaan Nonlinear . . . 30
4.1.1 Dari akar skalar ke beberapa variabel . . . 30
4.1.2 Penurunan Newton multivariabel . . . 31
4.1.3 Algoritma Newton multivariabel . . . 31
4.1.4 Contoh terhitung . . . 31
4.1.5 Pisahkan function dan driver . . . 32
4.1.6 Latihan SPNL . . . 33
4.2 Bagian B — Regresi linear dan nonlinear . . . 33
4.2.1 Data dan definisi dua model . . . 33
4.2.2 Regresi linear: dari OLS sampai SPL . . . 33
4.2.3 Regresi nonlinear: fungsi, Jacobian, dan pembaruan parameter . . . 34
4.2.4 Menurunkan algoritma Gauss–Newton . . . 35
4.2.5 Algoritma fungsi regresi nonlinear . . . 35
4.2.6 Evaluasi model dan grafik . . . 37
4.3 Latihan terintegrasi dari Lembar Kerja . . . 37
4.4 Tugas refleksi komputasi . . . 38

5 Praktikum 4: Interpolasi Polinomial 39
5.1 Dari data diskret ke fungsi . . . 39
5.2 Interpolasi sebagai Sistem Persamaan Linear . . . 40
5.3 Contoh 1: beberapa orde pada data yang sama . . . 40
5.4 Bentuk Newton dan divided differences . . . 40
5.5 Contoh 2: banyak titik dan pemilihan titik dasar . . . 41
5.6 Contoh 3: bentuk data yang lebih kompleks . . . 41

5.7 Function dan driver . . . 41
5.8 Banyak titik: jangan otomatis memakai satu polinom sangat tinggi . . . 44
5.9 Latihan di kelas . . . 44
5.10 Latihan mandiri . . . 45

6 Praktikum 5: Turunan Numerik dengan Beda Hingga 47
6.1 Materi singkat dan landasan teori . . . 47
6.2 Algoritma . . . 48
6.3 Contoh . . . 48
6.4 Program MATLAB: pisahkan function dan driver . . . 48
6.5 Luaran driver dan grafik galat . . . 49
6.6 Latihan . . . 49
6.7 Latihan terintegrasi dari Lembar Kerja . . . 50

7 Praktikum 6: Richardson, Studi Orde, dan Turunan pada Data 51
7.1 Dari beda hingga ke ide Richardson . . . 51
7.2 Memilih orde $p$ . . . 52
7.3 Rumus umum untuk masalah numerik lain . . . 52
7.4 Contoh dari materi kelas . . . 53
7.5 Dari satu titik ke banyak titik . . . 53
7.6 Program MATLAB: Richardson generik dan multi-titik . . . 53
7.7 Mini-project: dari data kumulatif penyakit menular ke laju kasus . . . 55
7.8 Latihan berorientasi project . . . 57
7.9 Latihan lanjutan dari Lembar Kerja . . . 58

8 Praktikum 7: Integral Numerik Dasar 59
8.1 Materi singkat dan landasan teori . . . 59
8.2 Algoritma . . . 60
8.3 Contoh . . . 60
8.4 Program MATLAB: pisahkan function dan driver . . . 60
8.5 Luaran driver dan grafik galat . . . 61
8.6 Latihan . . . 61
8.7 Latihan terintegrasi dari Lembar Kerja . . . 61

9 Praktikum 8: Titik Tengah dan Romberg 63
9.1 Materi singkat dan landasan teori . . . 63
9.2 Algoritma Romberg . . . 63

9.3 Contoh . . . 64
9.4 Program MATLAB: pisahkan function dan driver . . . 64
9.5 Luaran driver dan grafik galat . . . 65
9.6 Latihan . . . 65
9.7 Latihan lanjutan dari Lembar Kerja . . . 65

10 Praktikum 9: Masalah Nilai Awal - Euler dan Runge-Kutta 67
10.1 Materi singkat dan landasan teori . . . 67
10.2 Contoh model . . . 68
10.3 Algoritma . . . 68
10.4 Program MATLAB: pisahkan function dan driver . . . 69
10.5 Luaran driver dan grafik galat . . . 70
10.6 Latihan . . . 70
10.7 Latihan terintegrasi dari Lembar Kerja . . . 70

11 Praktikum 10: Stabilitas, Ukuran Langkah, dan Reproduksi Model PDB 71
11.1 Materi singkat dan landasan teori . . . 71
11.2 Contoh . . . 72
11.3 Program MATLAB: pisahkan function dan driver . . . 72
11.4 Luaran driver dan grafik galat . . . 73
11.5 Latihan . . . 73
11.6 Latihan lanjutan dari Lembar Kerja . . . 73

12 Praktikum 11: Praktikum Terpadu, Verifikasi, dan Validasi 75
12.1 Materi singkat: dari metode ke alur kerja . . . 75
12.2 Contoh terpadu: model logistik . . . 76
12.3 Algoritma alur kerja . . . 76
12.4 Program MATLAB: pisahkan function dan driver . . . 77
12.5 Latihan terpadu . . . 78

Mock-Up UTS dari Lembar Kerja 79
Soal Mock-Up UTS . . . 79

Mock-Up UAS dari Lembar Kerja 80
Soal Mock-Up UAS . . . 80

Rubrik Analisis Umum 81

Daftar File MATLAB 82

Referensi Utama 84

# Identitas Modul

| | |
|---|---|
| Mata kuliah | Pemodelan Komputasi dan Analisis Numerik |
| Kode | MA25-31017 |
| Program studi | Matematika |
| Fakultas | Fakultas Sains |
| Institusi | Institut Teknologi Sumatera |
| Perangkat lunak utama | MATLAB |
| Format pembelajaran | Praktikum, bedah jurnal, reproduksi komputasi, dan pembelajaran berbasis proyek |

## Tentang modul

Modul ini merupakan buku praktikum (bukan slide) untuk mata kuliah Pemodelan Komputasi dan Analisis Numerik. Struktur penyajian mengikuti pola modul praktikum: capaian, target luaran, materi singkat, algoritma, contoh, program MATLAB, verifikasi, dan latihan. Struktur praktikum terdiri atas sebelas pertemuan terhitung. Seluruh butir pada Lembar Kerja/Bank Soal Praktikum B0.1–B7.5, Mock-Up UTS, dan Mock-Up UAS diintegrasikan kembali ke dalam contoh atau latihan modul dengan ID yang dipertahankan agar sinkron. Landasan teori utama modul menggunakan Chapra dan Canale, Numerical Methods for Engineers, edisi ke-7.

## Capaian Pembelajaran Mata Kuliah (CPMK) utama

CPMK 6.1. Mahasiswa mampu memecahkan (C4) masalah optimasi termasuk (dan tidak terbatas pada) masalah program linier, masalah pengujian hipotesis dan masalah pada proses stokastik dengan memilih (C3) model matematika yang sesuai, menerapkan (C3) langkah-langkah metode numerik yang terukur dan valid secara analitik untuk mendapatkan solusi, serta menginterpretasikan solusi yang diperoleh sesuai konteks masalah.

CPMK 7.1. Mahasiswa mampu merancang, menerapkan (C3), dan menganalisis (C4) model matematika berbasis metode numerik atau komputasi untuk memecahkan masalah di bidang sains, teknik, ekonomi, atau industri, serta mengevaluasi (C5) keakuratan, keterbatasan, dan rel-

evansi model dalam konteks masalah nyata, dengan mendokumentasikan hasilnya secara ilmiah.

## Peta praktikum

| Praktik | Minggu | Indikator | Fokus |
|---|---|---|---|
| 1 | M2 | 6.1.1–6.1.3, 7.1.1 | Akar persamaan nonlinear: metode pengurungan dan metode terbuka |
| 2 | M3 | 6.1.4 | Sistem Persamaan Linear (SPL) iteratif: Jacobi, Gauss-Seidel, residual, konvergensi |
| 3 | M4 | 6.1.5, 7.1.1 | SPNL; regresi linear; pencocokan kurva nonlinear; MSE, RMSE, MAPE, dan $R^2$ |
| 5 | M6 | 6.1.6 | Turunan numerik: beda maju, beda mundur, beda pusat |
| 6 | M7 | 6.1.7, 7.1.1 | Richardson umum, studi orde, multi-titik, visualisasi, dan aplikasi data |
| 7 | M9 | 6.1.8 | Integral numerik: Trapezoid dan Simpson; pembanding Titik Tengah |
| 8 | M10 | 6.1.9, 7.1.1 | Titik Tengah, Romberg, studi akurasi dan reproduksi artikel |
| 9 | M11 | 6.1.10 | Masalah nilai awal: Euler dan evaluasi galat lokal/-global |
| 10 | M12 | 6.1.11, 7.1.1 | Runge-Kutta, stabilitas, ukuran langkah, reproduksi kurva model |
| 11 | M13 | 6.1/7.1 | Praktikum terpadu: pemilihan metode, verifikasi dan validasi |

## Peta ketercakupan Lembar Kerja

| Bagian Lembar Kerja | Lokasi dalam modul | Cakupan |
|---|---|---|
| B0.1–B0.5 | MATLAB Dasar | Variabel, kontrol alur, fungsi, grafik, galat, toleransi |
| B1.1–B1.5, B2.1–B2.5 | Praktikum 1 | Akar: grafik, Biseksi, Regula Falsi, Newton, Titik Tetap, Secant |
| B3.1–B3.5 | Praktikum 2 | SPL, solusi eksak, Jacobi, Gauss-Seidel, reaktor |
| B4.1–B4.5 | Praktikum 3 | Newton multivariabel, OLS, Gauss–Newton, MSE, RMSE, MAPE, $R^2$ |
| B5.1–B5.4 | Praktikum 5 | Turunan analitik dan beda hingga |
| B5.5 | Praktikum 6 | Ukuran langkah, Richardson, studi orde, multi-titik, dan aplikasi data |
| B6.1–B6.3 | Praktikum 7 | Integral eksak, Trapezoid, Simpson |
| B6.4–B6.5 | Praktikum 8 | Integral komposit, refinement, evaluasi galat |
| B7.1–B7.2 | Praktikum 9 | Euler dan Heun |
| B7.3–B7.5 | Praktikum 10 | RK4, solusi eksak, perbandingan metode |
| UTS.1–UTS.4 | Lampiran Mock-Up UTS | Akar dan SPL |
| UAS.1–UAS.4 | Lampiran Mock-Up UAS | Turunan, integral, dan PDB |

# Petunjuk Umum Praktikum

## Struktur setiap bab

Setiap bab memuat enam komponen wajib: (1) capaian pembelajaran, (2) materi/teori, (3) algoritma, (4) contoh, (5) program MATLAB, dan (6) latihan. Naskah ini menambahkan tiga komponen yang disarankan untuk pembelajaran komputasi: target luaran, checklist analisis, dan verifikasi dan validasi.

## Sinkronisasi dengan Lembar Kerja

Setiap soal pada Lembar Kerja Praktikum memiliki ID tetap (B0.1–B7.5, UTS.1–UTS.4, dan UAS.1–UAS.4). ID yang sama dipertahankan di modul. Dengan demikian, asisten dapat mengambil soal pre-test dari Lembar Kerja sementara mahasiswa dapat menemukan konsep, contoh, atau latihan yang relevan pada bab modul yang sesuai.

## Pola penulisan function dan driver

Setiap praktikum memakai dua jenis file. Function menyimpan algoritma umum yang menerima input dan mengembalikan hasil serta riwayat. Driver menyiapkan data, model, parameter, memanggil function, lalu membuat tabel dan visualisasi. Jalankan skrip driver dari folder `Modul_Praktikum/matlab`. Jangan menaruh `clear` di dalam function numerik. Notasi `err` dan `errs` menyatakan rasio desimal, bukan persen.

Konsistensi lintas bab: akar dan SPNL memakai grafik perubahan parameter/solusi terhadap iterasi; SPL memakai galat serta residual terhadap iterasi; turunan/integral/PDB memakai galat terhadap ukuran langkah atau jumlah grid ketika algoritma bukan iterasi akar. Selalu beri label sumbu dan toleransi yang dipakai.

## Prinsip pemrograman numerik

i. Pisahkan skrip utama dan fungsi numerik. Skrip utama memuat data, parameter, visualisasi, dan analisis; fungsi memuat algoritma.

ii. Hindari invers matriks eksplisit. Di MATLAB, gunakan operator \ untuk menyelesaikan sistem linear.

iii. Gunakan minimal dua kriteria berhenti ketika relevan: perubahan iterasi dan residual.

iv. Selalu tetapkan `maxit` untuk mencegah loop tak berhingga.

v. Laporkan satuan, toleransi, ukuran langkah, jumlah iterasi, residual, dan metrik galat.

vi. Pisahkan verifikasi (apakah algoritma diimplementasikan benar?) dari validasi (apakah model/hasil sesuai fenomena atau data?).

## Format ringkas laporan praktikum

i. Tujuan dan capaian yang diuji.

ii. Model/persamaan dan asumsi.

iii. Metode numerik serta algoritma/pseudocode.

iv. Implementasi MATLAB yang dapat direproduksi.

v. Verifikasi: kasus uji, residual, konvergensi, atau pembanding analitik.

vi. Hasil: tabel dan grafik dengan label lengkap.

vii. Analisis: pengaruh parameter numerik (toleransi, ukuran langkah, tebakan awal) dan keterbatasan.

viii. Kesimpulan singkat berbasis hasil, bukan hanya mengulang teori.

Bab 1

# MATLAB Dasar

Capaian pembelajaran praktikum
Mahasiswa mampu menggunakan MATLAB untuk membuat variabel, percabangan, pengulangan, fungsi, dan grafik sederhana serta memahami ukuran galat sebelum menerapkannya pada algoritma numerik.

Target luaran
Satu skrip MATLAB dasar yang memuat variabel, `if-else`, `for/while`, pemanggilan fungsi, tabel hasil, dan grafik sederhana.

## 1.1 Variabel dan operasi dasar

MATLAB dapat digunakan sebagai kalkulator numerik sekaligus bahasa pemrograman. Contoh paling dasar:

```
clear; clc; close all

a = 1;
b = 1.5;
tol = 1e-6;
maxit = 100;

x = 0:0.1:2;
y = x.^3 - x - 1;
```

Untuk operasi elemen-per-elemen pada vektor gunakan `.*`, `./`, dan `.^`.

## 1.2 Percabangan **`if-else`**

Percabangan digunakan ketika program harus mengambil keputusan.

```
f = @(x) x.^3 - x - 1;
```

```
if f(a)*f(b) < 0
    disp('Interval mengurung akar')
else
    disp('Belum ada perubahan tanda')
end
```

## 1.3 Pengulangan

### for

Gunakan `for` ketika jumlah pengulangan sudah diketahui.

```
for k = 1:5
    fprintf('Iterasi ke-%d\n',k)
end
```

### while

Gunakan `while` ketika pengulangan bergantung pada suatu kondisi.

```
err = 1;
k = 0;

while err > 1e-6 && k < 100
    k = k + 1;
    err = err/2;
end
```

## 1.4 Membuat fungsi

Fungsi dibuat agar algoritma dapat dipakai kembali.

```
% File: nilaiF.m
function y = nilaiF(x)
    y = x.^3 - x - 1;
end
```

Kemudian dapat dipanggil dengan

```
fx = nilaiF(1.25);
```

## 1.5 Menggambar grafik sederhana

Grafik berguna untuk melihat bentuk fungsi sebelum melakukan perhitungan numerik.

```
x = linspace(0.5,1.7,300);
y = x.^3 - x - 1;

plot(x,y,'LineWidth',1.5)
yline(0,'--')
grid on
xlabel('x')
ylabel('f(x)')
title('Grafik f(x) = x^3 - x - 1')
```

Untuk grafik konvergensi, gunakan `semilogy` jika galat turun beberapa orde besaran.

```
errs = [1e-1 4e-2 1.2e-2 3e-3 8e-4];
semilogy(1:numel(errs),errs,'o-')
grid on
xlabel('Iterasi')
ylabel('Galat')
title('Grafik konvergensi')
```

## 1.6 Pola kerja driver dan function

Driver adalah skrip yang menentukan masalah, data, toleransi, memanggil function, dan menggambar grafik. Function menerima input formal, menghitung, dan mengembalikan output tanpa menghapus workspace pengguna. Pisahkan keduanya dalam file `.m` berbeda; nama file function harus sesuai nama fungsi.

Contoh file `relative_error.m` berikut digunakan di berbagai bab:

```
function e = relative_error(xnew, xold)
%RELATIVE_ERROR Scaled relative change (dimensionless, NOT percent).
% Safe when xnew = 0; supports vectors and arrays elementwise.
e = abs(xnew-xold)./max(1,abs(xnew));
end
```

## 1.7 Galat: definisi sebelum latihan

Metode numerik menghasilkan hampiran. Karena itu ukuran galat harus didefinisikan sebelum digunakan.

Jika nilai sebenarnya $x^*$ diketahui, galat absolut adalah

$$E_t = |x^* - x_a|. \tag{1.1}$$

Galat relatif sebenarnya adalah

$$\varepsilon_t = \frac{|x^* - x_a|}{|x^*|}, \qquad x^* \neq 0. \tag{1.2}$$

Pada proses iterasi, nilai sebenarnya biasanya belum diketahui. Gunakan perubahan relatif berskala

$$e_a^{(k)} = \frac{|x_{\text{new}} - x_{\text{old}}|}{\max(1, |x_{\text{new}}|)}. \tag{1.3}$$

Bentuk ini tetap terdefinisi ketika $x_{\text{new}} = 0$ atau sangat dekat nol. Jika ingin dinyatakan sebagai persen, gunakan $100e_a^{(k)}\%$.

Untuk vektor,

$$e_a^{(k)} = \max_i \frac{|x_{i,\text{new}} - x_{i,\text{old}}|}{\max(1, |x_{i,\text{new}}|)}. \tag{1.4}$$

Implementasi MATLAB:

```
function e = relerr(xnew,xold)
    e = abs(xnew-xold)./max(1,abs(xnew));
end
```

## 1.8 Latihan MATLAB Dasar

i. Buat variabel `a`, `b`, `tol`, dan `maxit`, lalu tampilkan dengan `fprintf`.

ii. Buat vektor $x = 0 : 0.1 : 2$ dan hitung $f(x) = x^3 - x - 1$.

iii. Gunakan `if-else` untuk memeriksa apakah $f(a)f(b) < 0$.

iv. Gunakan `for` untuk menghitung kuadrat bilangan 1 sampai 10.

v. Gunakan `while` yang berhenti jika galat lebih kecil dari $10^{-6}$.

vi. Buat fungsi untuk $f(x) = x^3 - x - 1$.

vii. Buat fungsi `relerr.m` berdasarkan formula perubahan relatif berskala di atas.

viii. Gambar grafik $f(x) = x^3 - x - 1$ dan tambahkan garis $y = 0$.

ix. Buat grafik `semilogy` untuk suatu vektor `errs`.

## 1.9 Latihan fondasi dari Lembar Kerja

B0.1 (L1) Jelaskan apa yang dimaksud dengan metode numerik. Apa perbedaan utama antara solusi eksak/analitik dan solusi numerik?

B0.2 (L2) Nilai sebenarnya adalah $x^* = \sqrt{3}$ dan hampiran yang diperoleh $x_a = 1.73$. Hitung galat absolut dan galat relatif sebenarnya.

B0.3 (L3) Suatu iterasi menghasilkan urutan hampiran 2.0000, 1.7500, 1.7143, 1.7321. Hitung perubahan relatif antariterasi dan bandingkan formula biasa dengan formula berskala yang menggunakan $\max(1, |x_{\text{new}}|)$.

B0.4 (L4) Jelaskan mengapa dua hampiran dapat mempunyai galat absolut yang sama tetapi kualitasnya berbeda. Kaitkan dengan galat relatif.

B0.5 (L5) Jelaskan peran round-off error, truncation error, toleransi, dan jumlah iterasi maksimum dalam algoritma numerik.

Bab 2

# Praktikum 1: Akar Persamaan Nonlinear

Capaian pembelajaran praktikum
Mahasiswa mampu menerapkan Biseksi dan Regula Falsi sebagai metode pengurungan, memahami Titik Tetap, Newton-Raphson, dan Secant, serta mengevaluasi konvergensi menggunakan residual dan galat iterasi.

Target luaran
Kode contoh kelas untuk Biseksi dan Regula Falsi, tabel iterasi, dan grafik konvergensi. Latihan utama dikerjakan dengan Biseksi, Newton-Raphson, dan Secant.

## 2.1 Masalah pencarian akar

Tujuan pencarian akar adalah mencari $x$ sehingga

$$f(x) = 0. \tag{2.1}$$

Sebelum menghitung, gambar grafik untuk melihat posisi perubahan tanda dan memilih interval atau tebakan awal yang masuk akal.

Urutan metode dalam praktikum ini adalah:

i. Biseksi,

ii. Regula Falsi,

iii. Titik Tetap,

iv. Newton-Raphson,

v. Secant.

## 2.2 Biseksi

Jika $f$ kontinu pada $[a, b]$ dan

$$f(a)f(b) < 0, \tag{2.2}$$

maka sedikitnya satu akar berada di dalam interval.

Biseksi mengambil titik tengah

$$x_r = \frac{a+b}{2}. \tag{2.3}$$

Kemudian interval diperbarui:

$$\begin{cases} b \leftarrow x_r, & f(a)f(x_r) < 0, \\ a \leftarrow x_r, & f(a)f(x_r) > 0. \end{cases} \tag{2.4}$$

Galat iterasi yang dipakai:

$$e_a = \frac{|x_{r,\text{new}} - x_{r,\text{old}}|}{\max(1, |x_{r,\text{new}}|)}. \tag{2.5}$$

### Langkah algoritma

i. Pilih $a, b$ sehingga $f(a)f(b) < 0$.

ii. Hitung $x_r = (a+b)/2$.

iii. Hitung $f(x_r)$ dan $e_a$.

iv. Pilih interval baru berdasarkan tanda $f(a)f(x_r)$.

v. Ulangi sampai toleransi atau iterasi maksimum.

## 2.3 Regula Falsi

Regula Falsi tetap memakai interval yang mengurung akar, tetapi kandidat akar tidak lagi menggunakan titik tengah. Bangun garis yang melalui $(a, f(a))$ dan $(b, f(b))$. Titik potong garis tersebut dengan sumbu-$x$ adalah

$$x_r = b - \frac{f(b)(b-a)}{f(b)-f(a)} = \frac{af(b)-bf(a)}{f(b)-f(a)}. \tag{2.6}$$

Setelah $x_r$ diperoleh, aturan pemilihan interval sama dengan Biseksi:

$$\begin{cases} b \leftarrow x_r, & f(a)f(x_r) < 0, \\ a \leftarrow x_r, & f(a)f(x_r) > 0. \end{cases} \tag{2.7}$$

Jadi perbedaan inti kedua metode hanya terletak pada cara menghitung $x_r$.

## 2.4 Titik Tetap

Persamaan $f(x) = 0$ diubah menjadi

$$x = g(x), \tag{2.8}$$

kemudian dilakukan iterasi

$$x_{k+1} = g(x_k). \tag{2.9}$$

Contoh:

$$e^{-x} - x = 0 \qquad \Longrightarrow \qquad x = e^{-x}. \tag{2.10}$$

Syarat lokal sederhana untuk kecenderungan konvergen adalah

$$|g'(x^*)| < 1. \tag{2.11}$$

## 2.5 Newton-Raphson

Newton-Raphson menggunakan garis singgung pada titik $(x_k, f(x_k))$:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \tag{2.12}$$

Turunan harus ditentukan terlebih dahulu.

Contoh, untuk

$$f(x) = e^{-x} - x, \tag{2.13}$$

diperoleh

$$f'(x) = -e^{-x} - 1. \tag{2.14}$$

## 2.6 Secant

Secant tidak memerlukan turunan analitik:

$$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}. \tag{2.15}$$

## 2.7 Contoh kelas: Biseksi dan Regula Falsi

Gunakan fungsi yang sama:

$$f(x) = x^3 - x - 1, \qquad [a, b] = [1, 1.5]. \tag{2.16}$$

Karena $f(1) = -1$ dan $f(1.5) = 0.875$, interval mengurung akar.

Untuk Biseksi,

$$x_r = \frac{1 + 1.5}{2} = 1.25. \tag{2.17}$$

Untuk Regula Falsi,

$$x_r = 1.5 - \frac{0.875(1.5 - 1)}{0.875 - (-1)} \approx 1.266667. \tag{2.18}$$

Setelah nilai $x_r$ diperoleh, bagian kode yang memilih interval baru adalah identik.

## 2.8 Program MATLAB yang dibahas di kelas: function dan driver

Pisahkan kode sesuai contoh yang diberikan: function berisi algoritma dan mengembalikan akar beserta `history`, sedangkan driver menetapkan $f$, $a$, $b$, toleransi, menampilkan tabel, dan menggambar grafik. Tidak ada `clear`, data contoh, ataupun `plot` di dalam function.

`relative_error.m` menyimpan formula $|x_{new} - x_{old}|/\max(1, |x_{new}|)$ dalam skala desimal; gunakan `100*err` hanya bila benar-benar memberi label persen.

`biseksi_fun.m (function)`

```
function [x, history] = biseksi_fun(f, a, b, tol, maxit)
%BISEKSI_FUN Fungsi algoritma, tanpa clear/plot dan tanpa data hard-coded.
% Input: f, interval [a,b], tol, maxit. Output: root x, history.
if ~(isscalar(a)&&isscalar(b)&&a<b&&tol>0&&maxit>=1)
    error('Parameter interval/toleransi/iterasi tidak valid.');
end
fa=f(a); fb=f(b);
if fa == 0, x=a; history=table(0,a,0,0,'VariableNames',{'iter','x','err','residual
    '});return;end
if fb == 0, x=b; history=table(0,b,0,0,'VariableNames',{'iter','x','err','residual
    '});return;end
if ~isfinite(fa*fb) || fa*fb>0
    error('f(a) dan f(b) harus terhingga dan berlainan tanda.');
end
iter=zeros(maxit,1); xs=zeros(maxit,1); errs=inf(maxit,1); res=zeros(maxit,1);
xold=NaN;
for k=1:maxit
    x=(a+b)/2;                          % HANYA baris ini yang beda dari Regula
    fx=f(x);
    if ~isfinite(fx), error('Evaluasi fungsi tidak terhingga.'); end
    if k>1, err=relative_error(x,xold); else, err=Inf; end
    iter(k)=k; xs(k)=x; errs(k)=err; res(k)=abs(fx);
    if abs(fx)<=tol || (k>1 && err<=tol), break; end
    if fa*fx<0, b=x; fb=fx; elseif fa*fx>0, a=x; fa=fx; else, break; end
    xold=x;
end
history=table(iter(1:k),xs(1:k),errs(1:k),res(1:k),...
    'VariableNames',{'iter','x','err','residual'});
end
```

`regula_fun.m (function)`

```
function [x, history] = regula_fun(f, a, b, tol, maxit)
%REGULA_FUN Formula xr berbeda, pembaruan interval sama seperti Biseksi.
if ~(isscalar(a)&&isscalar(b)&&a<b&&tol>0&&maxit>=1)
    error('Parameter interval/toleransi/iterasi tidak valid.');
end
fa=f(a); fb=f(b);
if fa == 0, x=a; history=table(0,a,0,0,'VariableNames',{'iter','x','err','residual
    '});return;end
if fb == 0, x=b; history=table(0,b,0,0,'VariableNames',{'iter','x','err','residual
    '});return;end
if ~isfinite(fa*fb) || fa*fb>0
    error('f(a) dan f(b) harus terhingga dan berlainan tanda.');
end
iter=zeros(maxit,1); xs=zeros(maxit,1); errs=inf(maxit,1); res=zeros(maxit,1);
xold=NaN;
for k=1:maxit
    den=fb-fa;
    if abs(den)<eps*max(1,abs(fa)+abs(fb))
        error('Penyebut Regula Falsi terlalu kecil.');
    end
    x=b-fb*(b-a)/den;                   % HANYA baris ini yang beda dari Biseksi
```

```
    fx=f(x);
    if ~isfinite(fx), error('Evaluasi fungsi tidak terhingga.'); end
    if k>1, err=relative_error(x,xold); else, err=Inf; end
    iter(k)=k; xs(k)=x; errs(k)=err; res(k)=abs(fx);
    if abs(fx)<=tol || (k>1 && err<=tol), break; end
    if fa*fx<0, b=x; fb=fx; elseif fa*fx>0, a=x; fa=fx; else, break; end
    xold=x;
end
history=table(iter(1:k),xs(1:k),errs(1:k),res(1:k),...
    'VariableNames',{'iter','x','err','residual'});
end
```

`driver_akar.m (driver)`

```
%% DRIVER AKAR: siapkan fungsi, panggil dua algoritma, analisis hasil
clear; clc; close all; format long g
f = @(x) x.^3-x-1;
a=1; b=1.5; tol=1e-8; maxit=100;

[xB,hB] = biseksi_fun(f,a,b,tol,maxit);
[xR,hR] = regula_fun(f,a,b,tol,maxit);
fprintf('Biseksi: %.12f; residual %.3e\n',xB,abs(f(xB)));
fprintf('Regula : %.12f; residual %.3e\n',xR,abs(f(xR)));
disp(hB); disp(hR);

figure
semilogy(hB.iter,max(hB.err,eps),'o-',...
         hR.iter,max(hR.err,eps),'s-'); grid on
xlabel('Iterasi'); ylabel('Perubahan relatif berskala')
legend('Biseksi','Regula Falsi','Location','best')
title('Konvergensi akar')
```

Catatan numerik. Bandingkan kedua function baris demi baris: yang berubah adalah rumus $x_r$; pembaruan interval setelahnya identik. Newton-Raphson dan Secant dipakai sebagai tugas mandiri, bukan demo kode utama di kelas.

## 2.9 Latihan / tugas utama

i. Biseksi. Selesaikan $x^3 - 4x - 9 = 0$. Tentukan interval sendiri, tampilkan tabel iterasi, simpan `errs`, dan gambar grafik konvergensi.

ii. Newton-Raphson. Untuk $f(x) = e^{-x} - x$, tentukan $f'(x)$, gunakan sedikitnya dua tebakan awal, simpan `errs`, dan gambar grafik konvergensi.

iii. Secant. Gunakan fungsi yang sama dengan Newton-Raphson, pilih dua tebakan awal, simpan `errs`, lalu bandingkan jumlah iterasi dan pola grafiknya dengan Newton-Raphson.

## 2.10 Latihan terintegrasi dari Lembar Kerja

B1.1 (L1) Untuk $f(x) = x^3 - x - 1$, buat tabel nilai $f(x)$ untuk $x = 0.5, 1, 1.25, 1.5, 2$ dan tentukan interval yang masuk akal sebagai bracket.

B1.2 (L2) Gunakan Biseksi untuk $f(x) = x^3 - x - 1$ pada $[1, 1.5]$. Lakukan 4 iterasi dan tuliskan tabel iterasinya.

B1.3 (L3) Gunakan Regula Falsi untuk fungsi yang sama dengan $a = 1$, $b = 1.5$. Lakukan 5 iterasi.

B1.4 (L4) Gunakan Newton-Raphson untuk $f(x) = x^3 - x - 1$. Tentukan turunannya terlebih dahulu.

B1.5 (L5) Gunakan Newton-Raphson untuk $f(x) = e^{-x} - x$. Tentukan turunannya, lakukan 5 iterasi, dan bandingkan konvergensinya dengan B1.4.

B2.1 (L1) Untuk $e^{-x} - x = 0$, gunakan $g(x) = e^{-x}$ dan hitung 5 iterasi titik tetap dari $x_0 = 0$.

B2.2 (L2) Evaluasi $|g'(x^*)|$ di sekitar $x^* \approx 0.567$.

B2.3 (L3) Gunakan Secant untuk $f(x) = e^{-x} - x$ dengan $x_0 = 0$, $x_1 = 1$, dan lakukan 5 pembaruan.

B2.4 (L4) Untuk $\cos x - x = 0$, tuliskan dua kemungkinan bentuk titik tetap dan diskusikan kecenderungan konvergensinya.

B2.5 (L5) Bandingkan Biseksi, Regula Falsi, Titik Tetap, Newton-Raphson, dan Secant dari kebutuhan tebakan awal, turunan, kecepatan, dan risiko divergensi.

Bab 3

# Praktikum 2: Sistem Persamaan Linear Iteratif

Capaian pembelajaran praktikum
Mahasiswa mampu membangun algoritma Jacobi dan Gauss-Seidel dari bentuk persamaan skalar, kemudian menggunakan notasi matriks, mengevaluasi residual, galat, dan konvergensi.

Target luaran
Implementasi Jacobi dan Gauss-Seidel, tabel iterasi, solusi pembanding, residual, dan grafik `errs`.

## 3.1 Mulai dari persamaan, bukan matriks

Pertimbangkan tiga persamaan

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1, \tag{3.1}$$
$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2, \tag{3.2}$$
$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3. \tag{3.3}$$

Masing-masing persamaan diselesaikan terhadap variabel diagonalnya:

$$x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}, \tag{3.4}$$
$$x_2 = \frac{b_2 - a_{21}x_1 - a_{23}x_3}{a_{22}}, \tag{3.5}$$
$$x_3 = \frac{b_3 - a_{31}x_1 - a_{32}x_2}{a_{33}}. \tag{3.6}$$

Dari bentuk inilah Jacobi dan Gauss-Seidel dibangun.

## 3.2 Formula Jacobi

Jacobi menggunakan seluruh nilai dari iterasi lama:

$$x_1^{(k+1)} = \frac{b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)}}{a_{11}}, \tag{3.7}$$

$$x_2^{(k+1)} = \frac{b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)}}{a_{22}}, \tag{3.8}$$

$$x_3^{(k+1)} = \frac{b_3 - a_{31}x_1^{(k)} - a_{32}x_2^{(k)}}{a_{33}}. \tag{3.9}$$

Untuk $n$ variabel,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j^{(k)}\right). \tag{3.10}$$

## 3.3 Formula Gauss-Seidel

Gauss-Seidel langsung memakai nilai baru yang sudah tersedia:

$$x_1^{(k+1)} = \frac{b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)}}{a_{11}}, \tag{3.11}$$

$$x_2^{(k+1)} = \frac{b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)}}{a_{22}}, \tag{3.12}$$

$$x_3^{(k+1)} = \frac{b_3 - a_{31}x_1^{(k+1)} - a_{32}x_2^{(k+1)}}{a_{33}}. \tag{3.13}$$

Untuk $n$ variabel,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right]. \tag{3.14}$$

Perbedaan utama:

- Jacobi: semua komponen memakai iterasi $k$.

- Gauss-Seidel: komponen yang sudah dihitung langsung memakai iterasi $k+1$.

## 3.4 Baru kemudian notasi matriks

Sistem dapat diringkas menjadi

$$A\mathbf{x} = \mathbf{b}. \tag{3.15}$$

Notasi matriks berguna untuk implementasi umum, tetapi algoritma di atas tetap berasal dari persamaan skalar.

## 3.5 Galat iterasi dan residual

Gunakan

$$e_a^{(k)} = \max_i \frac{|x_i^{(k+1)} - x_i^{(k)}|}{\max(1, |x_i^{(k+1)}|)}. \tag{3.16}$$

Residual adalah

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}, \tag{3.17}$$

dan salah satu ukuran residual adalah

$$r_k = \|\mathbf{r}^{(k)}\|_\infty. \tag{3.18}$$

## 3.6 Contoh

Gunakan

$$10x_1 - x_2 + 2x_3 = 6, \tag{3.19}$$
$$-x_1 + 11x_2 - x_3 = 25, \tag{3.20}$$
$$2x_1 - x_2 + 10x_3 = -11. \tag{3.21}$$

Cari solusi eksak/analitik terlebih dahulu sebagai pembanding, lalu lakukan Jacobi dan Gauss-Seidel dari tebakan $(0, 0, 0)^T$.

## 3.7 Program MATLAB: function dan driver

Function `jacobi.m` dan `gauss_seidel.m` menerima $A, b, x_0, \tau, N_{\max}$ lalu mengembalikan solusi dan `history`. Driver `driver_spl.m` menyiapkan sistem dan solusi pembanding, memanggil kedua function, menampilkan tabel dan membuat grafik galat serta residual.

`jacobi.m (function)`

```
function [x, history] = jacobi(A, b, x0, tol, maxit)
%JACOBI Function: seluruh komponen baru memakai vector xold.
b=b(:); xold=x0(:); n=numel(b);
if ~isequal(size(A),[n n]) || numel(xold)~=n || tol<=0 || maxit<1
    error('Dimensi atau parameter tidak valid.');
end
if any(abs(diag(A))<=eps), error('Diagonal nol; susun ulang persamaan.'); end
xs=zeros(maxit,n); it=zeros(maxit,1); errs=zeros(maxit,1); res=zeros(maxit,1);
for k=1:maxit
    xnew=xold;
    for i=1:n
        idx=[1:i-1,i+1:n];
        xnew(i)=(b(i)-A(i,idx)*xold(idx))/A(i,i);
    end
    it(k)=k; xs(k,:)=xnew';
    errs(k)=max(relative_error(xnew,xold));
    res(k)=norm(b-A*xnew,inf);
    if errs(k)<=tol && res(k)<=tol, break; end
    xold=xnew;
end
x=xnew;
history=table(it(1:k),xs(1:k,:),errs(1:k),res(1:k),...
    'VariableNames',{'iter','x','err','residual'});
end
```

`gauss_seidel.m (function)`

```
function [x, history] = gauss_seidel(A, b, x0, tol, maxit)
%GAUSS_SEIDEL Function: komponen baru segera dipakai pada iterasi sama.
b=b(:); xold=x0(:); n=numel(b);
if ~isequal(size(A),[n n]) || numel(xold)~=n || tol<=0 || maxit<1
    error('Dimensi atau parameter tidak valid.');
end
if any(abs(diag(A))<=eps), error('Diagonal nol; susun ulang persamaan.'); end
xs=zeros(maxit,n); it=zeros(maxit,1); errs=zeros(maxit,1); res=zeros(maxit,1);
for k=1:maxit
    xnew=xold;
    for i=1:n
        left=A(i,1:i-1)*xnew(1:i-1);
        right=A(i,i+1:n)*xold(i+1:n);
        xnew(i)=(b(i)-left-right)/A(i,i);
    end
    it(k)=k; xs(k,:)=xnew';
    errs(k)=max(relative_error(xnew,xold));
    res(k)=norm(b-A*xnew,inf);
    if errs(k)<=tol && res(k)<=tol, break; end
    xold=xnew;
end
```

```
x=xnew;
history=table(it(1:k),xs(1:k,:),errs(1:k),res(1:k),...
    'VariableNames',{'iter','x','err','residual'});
end
```

`driver_spl.m (driver)`

```
%% DRIVER SPL: data, pemanggilan function, tabel, grafik
clear; clc; close all; format long g
A=[10 -1 2;-1 11 -1;2 -1 10]; b=[6;25;-11];
x0=zeros(3,1); tol=1e-8; maxit=100;
xexact=A\b;
[xj,hj]=jacobi(A,b,x0,tol,maxit);
[xg,hg]=gauss_seidel(A,b,x0,tol,maxit);
disp('Solusi eksak, Jacobi, Gauss-Seidel:');disp([xexact,xj,xg]);
disp(hj);disp(hg);
figure
semilogy(hj.iter,max(hj.err,eps),'o-',...
         hg.iter,max(hg.err,eps),'s-'); grid on
xlabel('Iterasi');ylabel('Galat iterasi berskala')
legend('Jacobi','Gauss-Seidel','Location','best')
title('Konvergensi SPL')
figure
semilogy(hj.iter,max(hj.residual,eps),'o-',...
         hg.iter,max(hg.residual,eps),'s-'); grid on
xlabel('Iterasi');ylabel('Norma residual infinity')
legend('Jacobi','Gauss-Seidel','Location','best')
```

> Catatan numerik. Jacobi membaca seluruh komponen `xold`; Gauss-Seidel membaca komponen kiri dari `xnew` dan kanan dari `xold`. Input dipaksa menjadi vektor kolom agar dimensi perhitungan konsisten.

## 3.8 Grafik konvergensi

Simpan galat setiap iterasi pada vektor `errs`.

```
semilogy(1:numel(errs),errs,'o-')
grid on
xlabel('Iterasi')
ylabel('e_a')
title('Grafik konvergensi')
```

Jika membandingkan Jacobi dan Gauss-Seidel, gambarkan kedua kurva pada satu grafik.

## 3.9 Latihan

i. Turunkan formula iterasi dari sistem persamaan, jangan mulai dari notasi matriks.

ii. Tentukan solusi eksak/analitik sebagai pembanding.

iii. Lakukan Jacobi dan Gauss-Seidel.

iv. Hitung galat iterasi dan residual.

v. Simpan sejarah galat pada `errs`.

vi. Buat grafik konvergensi kedua metode.

vii. Jelaskan metode mana yang lebih cepat pada kasus tersebut.

## 3.10 Latihan terintegrasi dari Lembar Kerja

B3.1 (L1) Tuliskan sistem $10x_1 - x_2 + 2x_3 = 6$, $-x_1 + 11x_2 - x_3 = 25$, $2x_1 - x_2 + 10x_3 = -11$ ke bentuk $A\mathbf{x} = \mathbf{b}$.

B3.2 (L2) Tentukan solusi eksak/analitik SPL tersebut.

B3.3 (L3) Gunakan Jacobi dari $(0, 0, 0)^T$, lakukan 3 iterasi, dan hitung galat setiap komponen.

B3.4 (L4) Gunakan Gauss-Seidel dari tebakan yang sama, lakukan 3 iterasi, lalu bandingkan dengan solusi eksak.

B3.5 (L5) Untuk model tiga reaktor, jelaskan pembentukan SPL dari neraca massa, susun bentuk matriksnya, dan tentukan solusi eksaknya.

Bab 4

# Praktikum 3: SPNL dan Pencocokan Kurva

Capaian pembelajaran praktikum
Mahasiswa mampu menurunkan dan menjalankan Newton untuk beberapa variabel, merumuskan OLS linear sebagai SPL, membentuk Jacobian model nonlinear, menjalankan Gauss–Newton, dan mengevaluasi hasil dengan galat serta grafik konvergensi.

Target luaran
Dua bagian terpisah: SPNL dan pencocokan kurva. Kode terdiri atas fungsi algoritma yang dapat digunakan ulang dan driver berisi data, parameter, tabel, serta visualisasi.

## 4.1 Bagian A — Newton untuk Sistem Persamaan Nonlinear

### 4.1.1 Dari akar skalar ke beberapa variabel

Pada satu variabel, Newton mencari akar $f(x) = 0$ dengan menggunakan turunan $f'(x)$. Pada $n$ variabel, kita mencari vektor $\mathbf{x} = (x_1, \ldots, x_n)^T$ yang membuat seluruh persamaan bernilai nol:

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{x}) \end{bmatrix} = \mathbf{0}. \tag{4.1}$$

Turunan $f'(x)$ digantikan oleh matriks Jacobian, yaitu kumpulan turunan parsial:

$$J_F(\mathbf{x}) = \left[ \frac{\partial f_i}{\partial x_j}(\mathbf{x}) \right]_{i,j=1}^{n}. \tag{4.2}$$

Jangan mencampurkan nama matriks Jacobian ($J_F$) dengan metode Jacobi pada SPL: keduanya berbeda.

### 4.1.2 Penurunan Newton multivariabel

Linearisasi Taylor orde pertama di titik $\mathbf{x}^{(k)}$ menghasilkan

$$\mathbf{F}(\mathbf{x}^{(k)} + \Delta\mathbf{x}) \approx \mathbf{F}(\mathbf{x}^{(k)}) + J_F(\mathbf{x}^{(k)})\Delta\mathbf{x}. \tag{4.3}$$

Kita meminta model linear ini sama dengan nol. Akibatnya, setiap iterasi Newton menyelesaikan SPL:

$$J_F(\mathbf{x}^{(k)})\Delta\mathbf{x}^{(k)} = -\mathbf{F}(\mathbf{x}^{(k)}), \tag{4.4}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}^{(k)}. \tag{4.5}$$

Di MATLAB gunakan `delta = -J(z)\F(z)`, bukan menghitung `inv(J)`.

### 4.1.3 Algoritma Newton multivariabel

i. Tentukan $\mathbf{F}$, Jacobian $J_F$, tebakan awal $\mathbf{x}^{(0)}$, toleransi $\tau$, dan batas iterasi $N_{\max}$.

ii. Pada iterasi $k$, hitung $\mathbf{F}_k = \mathbf{F}(\mathbf{x}^{(k)})$ dan $J_k = J_F(\mathbf{x}^{(k)})$.

iii. Pastikan Jacobian tidak singular/nyaris singular; selesaikan SPL $J_k\Delta\mathbf{x}^{(k)} = -\mathbf{F}_k$.

iv. Perbarui $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}^{(k)}$. Jika langkah penuh memperbesar residual, coba langkah teredam atau ubah tebakan awal.

v. Simpan galat perubahan relatif berskala dan residual:

$$e_k = \max_i \frac{|x_i^{(k+1)} - x_i^{(k)}|}{\max(1, |x_i^{(k+1)}|)}, \qquad r_k = \|\mathbf{F}(\mathbf{x}^{(k+1)})\|_\infty. \tag{4.6}$$

vi. Hentikan jika kriteria yang dipilih terpenuhi; laporkan kedua ukuran dan status bila $N_{\max}$ tercapai.

Catatan numerik. Tebakan awal adalah educated guess, bukan harus angka tertentu. Patuhi domain model (misalnya konsentrasi positif), lihat grafik/arti variabel, dan hindari tebakan pada Jacobian singular. Newton lokal tidak menjamin konvergensi dari sebarang tebakan.

### 4.1.4 Contoh terhitung

Tentukan penyelesaian

$$f_1(x, y) = x^2 + y^2 - 5 = 0, \tag{4.7}$$

$$f_2(x, y) = x - y - 1 = 0. \tag{4.8}$$

Dengan $\mathbf{x}^{(0)} = (1.5, 0.5)^T$, diperoleh

$$\mathbf{F}(\mathbf{x}^{(0)}) = \begin{bmatrix} -2.5 \\ 0 \end{bmatrix}, \qquad J_F(\mathbf{x}^{(0)}) = \begin{bmatrix} 3 & 1 \\ 1 & -1 \end{bmatrix}. \tag{4.9}$$

Selesaikan $J_F \Delta\mathbf{x} = -\mathbf{F}$ untuk memperoleh $\Delta\mathbf{x} = (0.625, 0.625)^T$; jadi $\mathbf{x}^{(1)} = (2.125, 1.125)^T$. Lanjutkan sampai residual kecil. Periksa solusi analitik $(2, 1)$ dan $(-1, -2)$: sistem ini mempunyai lebih dari satu akar.

### 4.1.5 Pisahkan function dan driver

Function: `newton_spnl.m` memuat algoritma dan `history`. Driver: `driver_spnl.m` menetapkan persamaan, Jacobian, tebakan awal, tabel, serta grafik.

`newton_spnl.m (function)`

```
function [x, history] = newton_spnl(F, Jac, x0, tol, maxit)
%NEWTON_SPNL Newton multivariabel; F dan Jac diberikan dari driver.
x=x0(:); n=numel(x);
if tol<=0 || maxit<1, error('Toleransi/maxit harus positif.'); end
it=zeros(maxit,1); xs=zeros(maxit,n); errs=zeros(maxit,1); res=zeros(maxit,1);
for k=1:maxit
    fk=F(x); Jk=Jac(x);
    if numel(fk)~=n || ~isequal(size(Jk),[n,n]) || ~all(isfinite(Jk(:)))
        error('Ukuran/nilai F atau Jacobian tidak sesuai.');
    end
    if rcond(Jk)<1e-12
        error('Jacobian singular/hampir singular: ganti tebakan awal.');
    end
    dx=-Jk\fk(:);
    xnew=x+dx;
    it(k)=k; xs(k,:)=xnew';
    errs(k)=max(relative_error(xnew,x));
    res(k)=norm(F(xnew),inf);
    if ~isfinite(res(k)), error('Residual tidak terhingga.'); end
    x=xnew;
    if errs(k)<=tol && res(k)<=tol, break; end
end
history=table(it(1:k),xs(1:k,:),errs(1:k),res(1:k),...
    'VariableNames',{'iter','x','err','residual'});
end
```

`driver_spnl.m (driver)`

```
%% DRIVER SPNL: model, Jacobian, tebakan, hasil, visualisasi
clear; clc; close all; format long g
F=@(z)[z(1)^2+z(2)^2-5; z(1)-z(2)-1];
Jac=@(z)[2*z(1), 2*z(2); 1, -1];
x0=[1.5; 0.5]; tol=1e-10; maxit=50;
[x,h]=newton_spnl(F,Jac,x0,tol,maxit);
fprintf('x=%.12f, y=%.12f, residual=%.3e\n',x(1),x(2),norm(F(x),inf));
disp(h)
figure
```

```
semilogy(h.iter,max(h.err,eps),'o-',...
         h.iter,max(h.residual,eps),'s-');grid on
xlabel('Iterasi');ylabel('Galat / residual')
legend('errs','residual','Location','best')
title('Konvergensi Newton SPNL')
```

### 4.1.6 Latihan SPNL

i. Turunkan $J_F$ sendiri sebelum menjalankan kode; hitung satu iterasi manual.

ii. Jalankan dari tiga tebakan awal dan identifikasi solusi yang diperoleh; perhatikan domain yang relevan.

iii. Buat `semilogy` untuk `history.errs` dan `history.residual`; jelaskan jika suatu grafik stagnan.

iv. Modifikasi driver untuk sistem lain, sedangkan function Newton tetap sama.

## 4.2 Bagian B — Regresi linear dan nonlinear

### 4.2.1 Data dan definisi dua model

Pada contoh kelas, gunakan data yang sama untuk kedua model:

| | | | | | |
|---|---|---|---|---|---|
| $x_i$ | 0.25 | 0.75 | 1.25 | 1.75 | 2.25 |
| $y_i$ | 0.28 | 0.57 | 0.68 | 0.74 | 0.79 |

Model linear: $\widehat{y}_i = a + bx_i$. Model nonlinear (saturasi sesuai contoh kelas):

$$\widehat{y}_i = f(x_i;\mathbf{a}) = a_0(1 - e^{-a_1 x_i}), \qquad \mathbf{a} = (a_0, a_1)^T, \quad \mathbf{a}^{(0)} = (1,1)^T. \tag{4.10}$$

Model linear mempunyai koefisien linear dalam parameter. Model saturasi nonlinear dalam parameter $a_1$, sehingga membutuhkan iterasi untuk mendapatkan nilai least squares.

### 4.2.2 Regresi linear: dari OLS sampai SPL

Residual $r_i = y_i - (a + bx_i)$. Ordinary least squares (OLS) meminimalkan

$$S(a,b) = \sum_{i=1}^{n} [y_i - a - bx_i]^2. \tag{4.11}$$

Setel kedua turunan parsial objektif ke nol:

$$\frac{\partial S}{\partial a} = -2\sum_i (y_i - a - bx_i) = 0, \tag{4.12}$$

$$\frac{\partial S}{\partial b} = -2\sum_i x_i(y_i - a - bx_i) = 0. \tag{4.13}$$

Diperoleh SPL normal equations:

$$\underbrace{\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix}}_{X^TX} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{\boldsymbol{\beta}} = \underbrace{\begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}}_{X^T\mathbf{y}}, \quad X = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}. \tag{4.14}$$

Untuk data kelas,

$$\begin{bmatrix} 5 & 6.25 \\ 6.25 & 10.3125 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 3.06 \\ 4.42 \end{bmatrix}, \qquad \widehat{\boldsymbol{\beta}} = \begin{bmatrix} 0.3145 \\ 0.2380 \end{bmatrix}. \tag{4.15}$$

Dalam perangkat lunak, `X\y` menyelesaikan OLS lebih stabil daripada `inv(X'*X)*X'*y`; persamaan normal dipakai untuk memahami formulasi, bukan perintah wajib menghitung invers.

`regresi_linear.m (function)`

```
function [beta, info] = regresi_linear(xdata, ydata)
%REGRESI_LINEAR OLS y=a+b*x via QR-backed backslash X\y (not inverse).
x=xdata(:); y=ydata(:);
if numel(x)~=numel(y) || numel(x)<3 || ~all(isfinite([x;y]))
    error('Data x,y harus berpasangan, terhingga, dan minimal 3 titik.');
end
X=[ones(numel(x),1),x];
if rank(X)<2, error('Variasi x tidak cukup; matriks desain rank-deficient.');end
beta=X\y;
info.X=X; info.normalA=X'*X; info.normalB=X'*y;
info.yhat=X*beta; info.residual=y-info.yhat;
end
```

### 4.2.3 Regresi nonlinear: fungsi, Jacobian, dan pembaruan parameter

Untuk model (4.10), residual dan objektifnya ialah

$$D_i(\mathbf{a}) = y_i - f(x_i;\mathbf{a}), \qquad S(\mathbf{a}) = \tfrac{1}{2}\sum_i D_i(\mathbf{a})^2. \tag{4.16}$$

Turunan parsial model terhadap parameter:

$$\frac{\partial f}{\partial a_0} = 1 - e^{-a_1 x}, \tag{4.17}$$

$$\frac{\partial f}{\partial a_1} = a_0 x e^{-a_1 x}. \tag{4.18}$$

Susun Jacobian model $Z \in \mathbb{R}^{n\times 2}$ dengan baris

$$Z_{i,:}(\mathbf{a}) = \begin{bmatrix} 1 - e^{-a_1 x_i} & a_0 x_i e^{-a_1 x_i} \end{bmatrix}. \tag{4.19}$$

Di $\mathbf{a}^{(0)} = (1, 1)^T$ berlaku (pembulatan empat desimal):

$$Z_0 = \begin{bmatrix} 0.2212 & 0.1947 \\ 0.5276 & 0.3543 \\ 0.7135 & 0.3581 \\ 0.8262 & 0.3041 \\ 0.8946 & 0.2371 \end{bmatrix}, \qquad \mathbf{D}_0 = \begin{bmatrix} 0.0588 \\ 0.0424 \\ -0.0335 \\ -0.0862 \\ -0.1046 \end{bmatrix}. \tag{4.20}$$

Ini adalah matriks dan residual pada gambar contoh kelas; nilai lengkap tanpa pembulatan digunakan dalam program.

### 4.2.4 Menurunkan algoritma Gauss–Newton

Linearisasi model $f(\mathbf{a} + \Delta\mathbf{a}) \approx f(\mathbf{a}) + Z\Delta\mathbf{a}$ menghasilkan

$$\mathbf{D}(\mathbf{a} + \Delta\mathbf{a}) \approx \mathbf{D} - Z\Delta\mathbf{a}. \tag{4.21}$$

Cari $\Delta\mathbf{a}$ yang meminimalkan $\|\mathbf{D} - Z\Delta\mathbf{a}\|_2^2$. Persamaan normal untuk langkah adalah

$$\underbrace{(Z^T Z)}_{2\times 2}\Delta\mathbf{a} = Z^T\mathbf{D}, \tag{4.22}$$

lalu perbarui parameter:

$$\mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} + \alpha_k\Delta\mathbf{a}^{(k)}, \qquad 0 < \alpha_k \leq 1. \tag{4.23}$$

Secara numerik, implementasi `delta = Z\D` memecahkan masalah least squares tanpa membentuk $Z^T Z$ secara eksplisit. `alpha` dapat dibagi dua bila objektif justru membesar. Untuk data contoh, langkah penuh pertama sekitar $\Delta\mathbf{a} = (-0.27148,\, 0.50193)^T$.

### 4.2.5 Algoritma fungsi regresi nonlinear

i. Input $x, y$, model $f(x; \mathbf{a})$, Jacobian model $Z(x, \mathbf{a})$, tebakan awal, toleransi dan batas iterasi.

ii. Evaluasi vektor residual $\mathbf{D} = \mathbf{y} - \hat{\mathbf{y}}$ dan Jacobian $Z$ berukuran $n \times p$.

iii. Periksa dimensi dan rank, lalu selesaikan masalah least squares $Z\Delta\mathbf{a} \approx \mathbf{D}$ (ekuivalen dengan (4.22) bila rank penuh).

iv. Uji parameter $\mathbf{a} + \alpha\Delta\mathbf{a}$; kurangi $\alpha$ jika SSE membesar.

v. Catat parameter, `errs` dan SSE untuk grafik; ulangi sampai toleransi atau iterasi maksimum.

vi. Periksa kewajaran parameter, pola residual dan kualitas hasil; perubahan parameter kecil saja tidak membuktikan kecocokan data baik.

regresi_nonlinear.m (function)

```
function [param, history] = regresi_nonlinear(xdata, ydata, param0, model, Jac,
    tol, maxit)
%REGRESI_NONLINEAR Damped Gauss-Newton for nonlinear least squares.
% model(x,p) -> n predictions; Jac(x,p) -> n-by-number_of_parameters.
% Uses QR-based Z\D rather than explicit inv(Z'*Z).
x=xdata(:); y=ydata(:);param=param0(:);
n=numel(x);m=numel(param);
if numel(y)~=n || n<m || tol<=0 || maxit<1
    error('Dimensi data, parameter, tol, atau maxit tidak valid.');
end
if ~all(isfinite([x;y;param])), error('Data/parameter tidak terhingga.');end
it=zeros(maxit,1); ps=zeros(maxit,m); errs=zeros(maxit,1);
sse=zeros(maxit,1); steps=zeros(maxit,1);
for k=1:maxit
    yfit=model(x,param); yfit=yfit(:); D=y-yfit;
    Z=Jac(x,param);
    if ~isequal(size(Z),[n,m]) || numel(yfit)~=n
        error('Model harus nx1 dan Jacobian harus nxp.');
    end
    if rank(Z)<m, error('Jacobian parameter rank-deficient; ganti tebakan/model.')
        ;end
    delta=Z\D;                                  % GN: Z'*Z*delta=Z'*D
    oldSSE=sum(D.^2);alpha=1; accepted=false;
    for bt=1:24
        trial=param+alpha*delta;
        pred=model(x,trial); pred=pred(:);
        newSSE=sum((y-pred).^2);
        if isfinite(newSSE) && newSSE<=oldSSE+1e-14*max(1,oldSSE)
            accepted=true;break
        end
        alpha=alpha/2;
    end
    if ~accepted
        % At an optimum the step may no longer lower SSE in floating point.
        trial=param;newSSE=oldSSE;alpha=0;
    end
    it(k)=k;ps(k,:)=trial';
    errs(k)=max(relative_error(trial,param));
    sse(k)=newSSE;steps(k)=alpha;
    param=trial;
    if errs(k)<=tol || alpha==0,break;end
end
history=table(it(1:k),ps(1:k,:),errs(1:k),sse(1:k),steps(1:k),...
    'VariableNames',{'iter','param','errs','SSE','alpha'});
end
```

driver_regresi.m (driver)

```
%% DRIVER REGRESI: data kelas, OLS dan Gauss-Newton, metrik serta plot
clear;clc;close all;format long g
x=[.25;.75;1.25;1.75;2.25];
y=[.28;.57;.68;.74;.79];

[beta,ols]=regresi_linear(x,y);
ylin=ols.yhat;

% Model saturasi dan Jacobian model dari contoh kelas:
```

```
model=@(x,p) p(1)*(1-exp(-p(2)*x));
Jac=@(x,p) [1-exp(-p(2)*x),p(1)*x.*exp(-p(2)*x)];
p0=[1;1]; tol=1e-10;maxit=100;
Z0=Jac(x,p0);D0=y-model(x,p0);
disp('Jacobian awal Z0:');disp(Z0)
disp('Residual awal D0:');disp(D0)

[p,h]=regresi_nonlinear(x,y,p0,model,Jac,tol,maxit);
ynon=model(x,p);
ML=metrik_regresi(y,ylin);
MN=metrik_regresi(y,ynon);
fprintf('OLS a=%.8f b=%.8f\n',beta(1),beta(2));
fprintf('Nonlinear a0=%.8f a1=%.8f\n',p(1),p(2));
metrics=[ML.MSE ML.RMSE ML.MAPE ML.R2;MN.MSE MN.RMSE MN.MAPE MN.R2];
disp(array2table(metrics,'VariableNames',{'MSE','RMSE','MAPE','R2'},...
    'RowNames',{'Linear','Nonlinear'}));
disp(h)

xx=linspace(min(x),max(x),250)';
figure;plot(x,y,'ko','MarkerFaceColor','k');hold on
plot(xx,beta(1)+beta(2)*xx,'-','LineWidth',1.5)
plot(xx,model(xx,p),'--','LineWidth',1.5);grid on
xlabel('x');ylabel('y')
legend('Data','OLS linear','Gauss-Newton saturasi','Location','best')
title('Dua model pada data yang sama')
figure;plot(x,y-ylin,'o-',x,y-ynon,'s-');hold on
plot([min(x) max(x)],[0 0],'k:');grid on
xlabel('x');ylabel('Residual y-yhat')
legend('Linear','Nonlinear','Nol','Location','best')
figure;semilogy(h.iter,max(h.errs,eps),'o-');grid on
xlabel('Iterasi');ylabel('errs: perubahan relatif parameter')
title('Konvergensi Gauss-Newton')
```

### 4.2.6 Evaluasi model dan grafik

Dengan $r_i = y_i - \widehat{y}_i$, $\bar{y} = \frac{1}{n}\sum_i y_i$, dan $\mathrm{SSE} = \sum r_i^2$:

$$\mathrm{MSE} = \frac{1}{n}\sum_i r_i^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \tag{4.24}$$

$$\mathrm{MAPE} = \frac{100}{n}\sum_i \left|\frac{r_i}{y_i}\right|, \qquad R^2 = 1 - \frac{\mathrm{SSE}}{\sum_i (y_i - \bar{y})^2}. \tag{4.25}$$

MAPE klasik tidak terdefinisi untuk $y_i = 0$; $R^2$ tidak terdefinisi jika seluruh $y_i$ sama. Jangan menuliskan hasil metrik tanpa memeriksa kondisi tersebut. `driver_regresi.m` menghasilkan grafik titik data dan kedua model, grafik residual, serta `semilogy` galat parameter per iterasi.

## 4.3 Latihan terintegrasi dari Lembar Kerja

B4.1 (L1) Susun $x^2 + y^2 - 5 = 0$ dan $x - y - 1 = 0$ dalam bentuk vektor $\mathbf{F}(x, y) = \mathbf{0}$. Jelaskan hubungan dengan pencarian akar skalar.

B4.2 (L2) Tentukan Jacobian $J_F(x, y)$ dan daerah di mana Jacobian tidak invertibel.

B4.3 (L3) Mulai dari $(1.5, 0.5)$, kerjakan satu langkah Newton multivariabel secara manual, lalu cocokkan dengan `driver_spnl.m`.

B4.4 (L4) Sesuai Lembar Kerja, gunakan data $(1, 0.5), (2, 2.5), (3, 2.0), (4, 4.0), (5, 3.5)$. (a) Cari model regresi linear $\widehat{y} = a + bx$ dengan OLS, turunkan SPL normalnya. (b) Tentukan model nonlinear: pada data yang sama fit $\widehat{y} = a_0(1 - e^{-a_1 x})$, bentuk Jacobian parameter $Z$, lalu jalankan Gauss–Newton. (c) Hitung MSE, RMSE, MAPE, $R^2$ dan buat grafik data-model, residual serta `errs`. Bandingkan pula dengan contoh data kelas di bagian sebelumnya.

B4.5 (L5) Gunakan dua belas data saturasi pada tabel berikut. Model default adalah $\widehat{y} = a_0(1-e^{-a_1 x})$. Hitung parameter dan keempat metrik serta grafik `errs`. Untuk tantangan, coba model saturasi rasional $\widehat{y} = c_0 x/(c_1 + x)$ pada data yang sama; turunkan Jacobian parameter dan bandingkan metriknya. Jelaskan perbedaan bentuk model dan batasan parameter. Untuk mempertahankan butir asli B4.5, tuliskan juga objektif $S(a, b) = \sum_i (y_i - ae^{bx_i})^2$ dan kondisi optimalitas $\partial_a S = \partial_b S = 0$ untuk model eksponensial $ae^{bx}$.

B4.6 (L6) Gunakan fungsi $\widehat{y} = ae^{bx}$ pada latihan B4.5, gunakan data sintetis berbeda berikut:

| | | | | | | |
|---|---|---|---|---|---|---|
| $x$ | 0.5 | 1.0 | 1.5 | 2.0 | 2.5 | 3.0 |
| $y$ | 0.806 | 0.920 | 1.159 | 1.410 | 1.641 | 2.057 |
| $x$ | 3.5 | 4.0 | 4.5 | 5.0 | 5.5 | 6.0 |
| $y$ | 2.488 | 2.932 | 3.604 | 4.366 | 5.235 | 6.395 |

Sebagai tantangan lain, bandingkan dengan fungsi pangkat $f(x; c, d) = cx^d$ (untuk $x > 0$).

| | | | | | | |
|---|---|---|---|---|---|---|
| $x$ | 0.25 | 0.75 | 1.25 | 1.75 | 2.25 | 2.75 |
| $y$ | 0.165 | 0.440 | 0.612 | 0.771 | 0.859 | 0.945 |
| $x$ | 3.25 | 3.75 | 4.25 | 4.75 | 5.25 | 5.75 |
| $y$ | 0.988 | 1.044 | 1.065 | 1.097 | 1.102 | 1.118 |

## 4.4 Tugas refleksi komputasi

Jalankan seluruh driver dengan data sesuai instruksi. Jelaskan isi input/output tiap function, tunjukkan apa yang berubah ketika tebakan awal diganti, serta simpan grafik `errs` dan residual dengan label lengkap.

Bab 5

# Praktikum 4: Interpolasi Polinomial

Capaian pembelajaran praktikum
Mahasiswa mampu membangun polinom interpolasi dari data diskret, menghubungkannya dengan Sistem Persamaan Linear, dan mengevaluasi pengaruh orde serta pemilihan titik data.

Target luaran
Polinom interpolasi beberapa orde, tabel selisih terbagi Newton, residual pada titik data, grafik data–polinom, dan analisis error.

## 5.1 Dari data diskret ke fungsi

Diberikan titik

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n).$$

Polinom interpolasi berderajat paling tinggi $n$ ditulis

$$p_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n, \tag{5.1}$$

dan melalui seluruh $n+1$ titik data. Dua titik membentuk polinom linear, tiga titik membentuk polinom kuadratik, dan seterusnya.

## 5.2 Interpolasi sebagai Sistem Persamaan Linear

Substitusi seluruh titik ke $p_n(x)$ menghasilkan

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix}}_{\mathbf{a}} = \underbrace{\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}}_{\mathbf{y}}. \tag{5.2}$$

Dengan demikian, koefisien polinom diperoleh dari SPL $A\mathbf{a} = \mathbf{y}$. Setelah koefisien ditemukan, nilai di antara titik data dapat dihitung langsung dari $p_n(x)$.

## 5.3 Contoh 1: beberapa orde pada data yang sama

Gunakan delapan titik berikut.

| $x$ | 1 | 4 | 6 | 5 | 3 | 1.5 | 2.5 | 3.5 |
|---|---|---|---|---|---|---|---|---|
| $y$ | 0 | 1.3862944 | 1.7917595 | 1.6094379 | 1.0986123 | 0.4054641 | 0.9162907 | 1.2527630 |

Data tersebut berasal dari $y = \ln x$. Estimasikan nilai pada $x = 2$ dengan orde yang bertambah. Beberapa hasil awal adalah

$$p_1(2) = 0.462098, \qquad p_2(2) = 0.565844, \qquad p_3(2) = 0.628769.$$

Dengan data yang lebih banyak, program dapat membangun polinom orde $1, 2, \ldots, 7$ dan menampilkan perubahan error terhadap orde. Nilai referensi $\ln 2$ hanya digunakan untuk menilai kualitas pendekatan.

## 5.4 Bentuk Newton dan divided differences

Bentuk Newton ditulis

$$\begin{aligned} p_n(x) = {} & b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1) + \cdots \\ & + b_n(x - x_0)(x - x_1)\cdots(x - x_{n-1}). \end{aligned} \tag{5.3}$$

Koefisien dihitung secara rekursif dari selisih terbagi:

$$b_0 = f[x_0], \tag{5.4}$$

$$b_1 = f[x_1, x_0] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}, \tag{5.5}$$

$$b_2 = f[x_2, x_1, x_0] = \frac{f[x_2, x_1] - f[x_1, x_0]}{x_2 - x_0}. \tag{5.6}$$

Keuntungan bentuk Newton adalah orde polinom dapat dinaikkan dengan menambah satu suku baru tanpa menghitung seluruh bentuk polinom dari awal.

## 5.5 Contoh 2: banyak titik dan pemilihan titik dasar

Gunakan data

| $x$ | 1 | 2 | 3 | 5 | 7 | 8 |
|---|---|---|---|---|---|---|
| $y$ | 3 | 6 | 19 | 99 | 291 | 444 |

dan estimasikan $f(4)$ dengan polinom orde 1 sampai 4. Untuk setiap orde, pilih titik dasar yang paling dekat dengan $x = 4$. Bandingkan perubahan prediksi ketika orde bertambah. Jika suatu orde tertentu tiba-tiba menghasilkan hasil eksak atau hampir eksak, periksa kemungkinan bahwa data berasal dari polinom dengan orde tersebut.

## 5.6 Contoh 3: bentuk data yang lebih kompleks

Gunakan

| $x$ | 0 | 1 | 2 | 5.5 | 11 | 13 | 16 | 18 |
|---|---|---|---|---|---|---|---|---|
| $y$ | 0.5 | 3.134 | 5.3 | 9.9 | 10.2 | 9.35 | 7.2 | 6.2 |

untuk mengestimasi nilai di sekitar $x = 8$. Bandingkan orde $1, 2, 3$, dan 5. Untuk data seperti ini, tidak cukup hanya melihat nilai interpolasi di satu titik; gambar seluruh kurva karena polinom orde tinggi dapat berosilasi di antara titik data.

## 5.7 Function dan driver

Function pertama menyusun matriks Vandermonde dan menyelesaikan SPL. Function kedua membentuk tabel divided differences Newton. Driver membaca beberapa dataset, mencoba banyak orde, menyimpan error, dan menggambar kurva.

`interp_poly_spl.m`

```
function [a, yq, A, residual] = interp_poly_spl(x, y, xq)
%INTERP_POLY_SPL Interpolasi polinomial dalam basis 1,x,...,x^n.
%   Koefisien diperoleh dari SPL Vandermonde A*a = y.

x = x(:);
y = y(:);
n = numel(x) - 1;

if numel(y) ~= n+1
    error('x dan y harus memiliki panjang yang sama.');
end
if numel(unique(x)) ~= numel(x)
    error('Semua nilai x harus berbeda.');
end

A = zeros(n+1,n+1);
for j = 0:n
    A(:,j+1) = x.^j;
```

```
end

a = A\y;
residual = norm(A*a-y,inf);

xq = xq(:);
Vq = zeros(numel(xq),n+1);
for j = 0:n
    Vq(:,j+1) = xq.^j;
end
yq = Vq*a;
end
```

newton_dd.m

```
function [b, yq, tableDD] = newton_dd(x, y, xq)
%NEWTON_DD Newton divided-difference interpolation.
%   b berisi koefisien Newton dan tableDD tabel selisih terbagi.

x = x(:);
y = y(:);
n = numel(x);

if numel(y) ~= n
    error('x dan y harus memiliki panjang yang sama.');
end
if numel(unique(x)) ~= n
    error('Semua nilai x harus berbeda.');
end

tableDD = zeros(n,n);
tableDD(:,1) = y;

for j = 2:n
    for i = 1:n-j+1
        tableDD(i,j) = (tableDD(i+1,j-1)-tableDD(i,j-1)) / ...
                       (x(i+j-1)-x(i));
    end
end

b = tableDD(1,:).';

yq = zeros(size(xq));
for q = 1:numel(xq)
    val = b(1);
    term = 1;
    for k = 2:n
        term = term*(xq(q)-x(k-1));
        val = val + b(k)*term;
    end
    yq(q) = val;
end
end
```

driver_interpolasi.m

```
%% Praktikum Interpolasi Polinomial
% Contoh pemakaian function interp_poly_spl.m dan newton_dd.m
```

```
clear; clc; close all

%% DATASET A: ln(x), 8 titik
T = readtable(fullfile('data','data_ln_chapra.csv'));
x = T.x; y = T.y;
xq = 2;
yref = log(xq);

fprintf('DATASET A: ln(x), xq = %.2f\n',xq)
fprintf('Nilai referensi = %.8f\n\n',yref)

% Urutan titik sengaja dapat diubah untuk mengamati pengaruh pemilihan titik.
orders = 1:7;
yest = nan(size(orders));
err = nan(size(orders));

for p = orders
    m = p+1;
    xm = x(1:m);
    ym = y(1:m);
    [~,yest(p),~,~] = interp_poly_spl(xm,ym,xq);
    err(p) = abs(yest(p)-yref);
    fprintf('orde %d : %.8f | error | = %.3e\n',p,yest(p),err(p))
end

figure
semilogy(orders,err,'o-','LineWidth',1.4)
grid on
xlabel('Orde polinom')
ylabel('|error|')
title('Perubahan error terhadap orde polinom')

% Bandingkan beberapa kurva tanpa membuat gambar terlalu padat.
xx = linspace(min(x),max(x),400).';
figure; hold on
scatter(x,y,55,'filled','DisplayName','Data')
for p = [1 2 3 5 7]
    m = p+1;
    [~,yy] = interp_poly_spl(x(1:m),y(1:m),xx);
    plot(xx,yy,'LineWidth',1.2,'DisplayName',sprintf('orde %d',p))
end
plot(xx,log(xx),'k--','LineWidth',1.4,'DisplayName','ln(x)')
grid on
xlabel('x'); ylabel('y')
title('Interpolasi beberapa orde')
legend('Location','best')

%% DATASET B: data yang berasal dari suatu polinom
T = readtable(fullfile('data','data_poly_chapra.csv'));
x = T.x; y = T.y; xq = 4;

fprintf('\nDATASET B: estimasi pada xq = %.2f\n',xq)
for p = 1:4
    % pilih p+1 titik yang dekat dengan xq
    [~,idx] = sort(abs(x-xq));
    idx = sort(idx(1:p+1));
    [~,yq] = newton_dd(x(idx),y(idx),xq);
    fprintf('orde %d : %.8f\n',p,yq)
```

```
end

%% DATASET C: bentuk data naik lalu turun
T = readtable(fullfile('data','data_shape_chapra.csv'));
x = T.x; y = T.y; xq = 8;

figure
scatter(x,y,55,'filled'); hold on
xx = linspace(min(x),max(x),500).';
for p = [1 2 3 5]
    [~,idx] = sort(abs(x-xq));
    idx = sort(idx(1:p+1));
    [~,yy] = newton_dd(x(idx),y(idx),xx);
    plot(xx,yy,'LineWidth',1.2,'DisplayName',sprintf('orde %d',p))
end
grid on
xlabel('x'); ylabel('y')
title('Pengaruh orde dan pemilihan titik')
legend('Data','orde 1','orde 2','orde 3','orde 5','Location','best')
```

## 5.8 Banyak titik: jangan otomatis memakai satu polinom sangat tinggi

Semakin banyak data memungkinkan orde semakin tinggi, tetapi bukan berarti polinom dengan orde maksimum selalu menjadi pilihan terbaik. Periksa grafik, perubahan prediksi antarorde, residual pada titik data, serta sensitivitas terhadap urutan dan lokasi titik dasar. Untuk data yang panjang, pendekatan piecewise menggunakan polinom orde rendah merupakan alternatif alami.

Sebagai pengantar, untuk data

| $x$ | 3 | 4.5 | 7 | 9 |
|---|---|---|---|---|
| $y$ | 2.5 | 1 | 2.5 | 0.5 |

sebuah spline kubik menggunakan polinom kubik berbeda pada setiap interval dan mensyaratkan sambungan yang mulus. Syarat kontinuitas tersebut menghasilkan SPL untuk menentukan koefisien atau turunan pada titik sambung.

## 5.9 Latihan di kelas

i. Linear. Gunakan $(1, 2)$ dan $(3, 8)$ untuk menentukan $p_1(x)$ dan nilai $p_1(2)$.

ii. Kuadratik sebagai SPL. Gunakan tiga titik pertama dari dataset $\ln x$, yaitu $(1, 0)$, $(4, 1.3862944)$, dan $(6, 1.7917595)$. Susun matriks Vandermonde, hitung koefisien $p_2(x)$, lalu estimasikan $p_2(2)$. Hitung $\|A\mathbf{a} - \mathbf{y}\|_\infty$.

iii. Newton. Gunakan titik yang sama dengan soal 2. Buat tabel divided differences sampai orde dua dan verifikasi bahwa nilai di $x = 2$ sama dengan hasil basis monomial/SPL.

iv. Orde bertambah. Pada dataset $\ln x$, hitung estimasi $\ln 2$ untuk orde 1 sampai 5. Buat tabel berisi orde, nilai estimasi, perubahan prediksi, dan error terhadap $\ln 2$.

v. Grafik. Untuk soal 4, gambar data dan polinom orde 1, 2, 3, dan 5 dalam satu gambar. Jelaskan apa yang berubah ketika orde meningkat.

## 5.10 Latihan mandiri

vi. Gunakan seluruh delapan titik dataset $\ln x$. Jalankan orde 1 sampai 7. Buat grafik |error| terhadap orde dengan `semilogy`. Tentukan pada orde berapa peningkatan mulai kecil.

vii. Gunakan dataset

| $x$ | 1 | 2 | 3 | 5 | 7 | 8 |
|---|---|---|---|---|---|---|
| $y$ | 3 | 6 | 19 | 99 | 291 | 444 |

untuk menentukan $f(4)$ menggunakan Newton orde 1 sampai 4. Untuk setiap orde, pilih titik dasar yang paling dekat dengan $x = 4$. Jelaskan apa yang dapat disimpulkan mengenai orde polinom yang mungkin menghasilkan data tersebut.

viii. Pada dataset yang sama dengan soal 7, ubah urutan titik dasar secara sengaja menjadi kurang baik. Bandingkan proses konvergensi prediksi dengan urutan titik yang dekat terhadap $x = 4$. Tampilkan tabel dan grafik perubahan estimasi.

ix. Gunakan dataset

| $x$ | 0 | 1 | 2 | 5.5 | 11 | 13 | 16 | 18 |
|---|---|---|---|---|---|---|---|---|
| $y$ | 0.5 | 3.134 | 5.3 | 9.9 | 10.2 | 9.35 | 7.2 | 6.2 |

untuk mengestimasi $y(8)$ dengan orde 1, 2, 3, dan 5. Pilih titik dasar yang dekat dengan $x = 8$. Gambar semua polinom pada rentang data dan komentari adanya osilasi atau perilaku tidak wajar.

x. Gunakan data

| $x$ | 0 | 1.8 | 5 | 6 | 8.2 | 9.2 | 12 |
|---|---|---|---|---|---|---|---|
| $y$ | 26 | 16.415 | 5.375 | 3.5 | 2.015 | 2.54 | 8 |

untuk menentukan $y(3.5)$. Coba beberapa orde dan susunan titik dasar. Pilih hasil yang menurut Anda paling meyakinkan berdasarkan kestabilan prediksi dan grafik.

xi. Inverse interpolation. Diberikan

| $x$ | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|
| $y$ | 0.5 | 0.3333 | 0.25 | 0.2 | 0.1667 | 0.1429 |

carilah $x$ ketika $y = 0.23$. Gunakan interpolasi kubik untuk membentuk $p(x)$ kemudian selesaikan $p(x) - 0.23 = 0$ dengan salah satu metode pencarian akar yang telah dipelajari.

xii. Piecewise. Untuk data $(3, 2.5)$, $(4.5, 1)$, $(7, 2.5)$, $(9, 0.5)$, bandingkan satu polinom kubik global dengan interpolasi linear per interval. Kemudian tuliskan syarat tambahan yang diperlukan jika setiap interval diganti polinom kubik dan sambungan diharuskan kontinu pada fungsi, turunan pertama, dan turunan kedua.

Bab 6

# Praktikum 5: Turunan Numerik dengan Beda Hingga

Capaian pembelajaran praktikum
Mahasiswa mampu menerapkan beda maju, beda mundur, dan beda pusat serta menjelaskan orde akurasi turunan numerik (indikator 6.1.6).

Target luaran
Fungsi MATLAB generik untuk tiga skema beda hingga, tabel galat terhadap ukuran langkah, dan grafik log-log yang menunjukkan perbedaan orde akurasi.

## 6.1 Materi singkat dan landasan teori

Beda hingga diturunkan dari ekspansi Taylor. Untuk $h > 0$,

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + O(h^3), \tag{6.1}$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) + O(h^3). \tag{6.2}$$

Dari persamaan pertama diperoleh beda maju

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} = f'(x) + O(h). \tag{6.3}$$

Beda mundur juga berorde satu. Dengan mengurangkan dua ekspansi Taylor diperoleh beda pusat

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} = f'(x) + O(h^2). \tag{6.4}$$

Karena itu, ketika $h$ dibagi dua, galat pemotongan beda maju idealnya turun sekitar faktor 2, sedangkan beda pusat sekitar faktor 4, selama pembulatan belum dominan.

Ukuran langkah tidak boleh dipilih sekecil mungkin tanpa batas. Ketika $h$ terlalu kecil, pengurangan dua bilangan yang hampir sama menyebabkan pembatalan numerik dan pembulatan

galat dapat membesar. Secara praktis terdapat wilayah dominasi galat pemotongan, wilayah optimum, dan wilayah dominasi pembulatan.

## 6.2 Algoritma

i. Tetapkan $f$, titik $x_0$, dan daftar $h$.

ii. Untuk setiap $h$, hitung beda maju, mundur, dan pusat.

iii. Jika solusi analitik tersedia, hitung galat absolut/relatif. Jika tidak, gunakan solusi referensi berakurasi lebih tinggi secara hati-hati.

iv. Plot galat terhadap $h$ pada skala log-log.

v. Estimasikan kemiringan grafik untuk mengamati orde akurasi.

## 6.3 Contoh

Ambil

$$f(x) = \frac{e^x}{\sin(\sqrt{x})}, \qquad x_0 = 1.2. \tag{6.5}$$

Turunan analitik pada $x_0$ sekitar 2.85683778241. Dengan $h = 0.1$, beda pusat memberikan sekitar 2.86120081209, jauh lebih akurat daripada beda maju/mundur pada step yang sama. Ketika $h$ dibagi dua berulang kali, galat beda pusat menunjukkan orde mendekati dua sebelum efek pembulatan muncul.

## 6.4 Program MATLAB: pisahkan function dan driver

Driver: parameter/data dan grafik. Function: algoritma numerik yang menerima input dan mengembalikan output. Semua file berada pada `Modul_Praktikum/matlab`.

`numdiff.m (function)`

```
function d = numdiff(f,x0,method,h)
%NUMDIFF Turunan pertama dengan beberapa skema beda hingga.
% x0 boleh skalar atau vektor jika f mendukung operasi elementwise.
if nargin < 4 || isempty(h)
    h = eps^(1/3)*max(1,max(abs(x0(:))));
end

switch lower(method)
    case 'forward'
        d = (f(x0+h)-f(x0))/h;
    case 'backward'
        d = (f(x0)-f(x0-h))/h;
    case 'central'
```

```
        d = (f(x0+h)-f(x0-h))/(2*h);
    case {'central4','central-4','fourth'}
        d = (f(x0-2*h)-8*f(x0-h)+8*f(x0+h)-f(x0+2*h))/(12*h);
    otherwise
        error('Metode: forward, backward, central, atau central4.');
end
end
```

`driver_turunan.m (driver)`

```
%% Praktikum 4: beda hingga dan orde akurasi
clear; clc; format long g
f=@(x) exp(x)./sin(sqrt(x)); x0=1.2;
s=sqrt(x0);
dexact=f(x0).*(1-cot(s)./(2*s));
hs=0.2./2.^(0:7);
for k=1:numel(hs)
    h=hs(k);
    df= numdiff(f,x0,'forward',h);
    db= numdiff(f,x0,'backward',h);
    dc= numdiff(f,x0,'central',h);
    tab(k,:)=[h,df,abs(df-dexact),db,abs(db-dexact),dc,abs(dc-dexact)];
end
disp(array2table(tab,'VariableNames',{'h','forward','errF','backward','errB','
    central','errC'}));
figure; loglog(hs,tab(:,3),'o-',hs,tab(:,5),'s-',hs,tab(:,7),'^-'); grid on
xlabel('h'); ylabel('error absolut'); legend('Forward','Backward','Central');
```

Checklist analisis. Jangan hanya melaporkan nilai turunan. Tunjukkan hubungan $h$-galat dan jawab: skema mana paling akurat pada biaya evaluasi fungsi yang sebanding?

## 6.5 Luaran driver dan grafik galat

Jalankan driver dan tampilkan keluaran numerik, nilai acuan bila tersedia, serta grafik berlabel lengkap. Plot galat terhadap ukuran langkah $h$ atau banyak subinterval $n$; simpan nilainya sebagai `errs` pada driver bila diperlukan.

## 6.6 Latihan

i. Turunkan sendiri formula beda pusat dari deret Taylor.

ii. Uji $f(x) = \sin(1/x)$ pada $x = 0.4$ untuk beberapa $h$. Jelaskan mengapa fungsi yang berosilasi cepat menuntut kehati-hatian.

iii. Estimasikan orde observasi dengan $p \approx \log(E_h/E_{h/2})/\log 2$.

iv. Buat fungsi untuk hampiran turunan kedua dengan beda pusat dan uji pada $f(x) = e^x$.

v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.

vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.

vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.

## 6.7 Latihan terintegrasi dari Lembar Kerja

Untuk setiap fungsi, tentukan turunan analitik terlebih dahulu agar galat sebenarnya dapat dievaluasi.

B5.1 (L1) Untuk $f(x) = x\sqrt{x} - 7x$, tentukan turunan analitik $f'(x)$.

B5.2 (L2) Gunakan beda maju, beda mundur, dan beda pusat dengan $h = 0.5$ untuk menghampiri $f'(1.5)$ pada fungsi B5.1.

B5.3 (L3) Hitung nilai eksak $f'(1.5)$ dari hasil soal B5.1, kemudian tentukan galat absolut dan galat relatif sebenarnya untuk ketiga hampiran pada B5.2.

B5.4 (L4) Untuk fungsi $f(x) = \ln x$, tentukan turunan analitik $f'(x)$, lalu hampiri $f'(2)$ dengan beda pusat menggunakan $h = 0.25$. Evaluasi galat relatif sebenarnya.

Bab 7

# Praktikum 6: Richardson, Studi Orde, dan Turunan pada Data

Capaian pembelajaran praktikum
Mahasiswa mampu menganalisis pengaruh ukuran langkah dan orde galat pada turunan numerik, menerapkan ekstrapolasi Richardson pada beberapa orde skema dasar dan banyak titik evaluasi, serta menginterpretasikan turunan dari model hasil fitting pada data tabel (indikator 6.1.7 dan 7.1.1).

Target luaran
Eksperimen yang membandingkan beda hingga biasa dengan Richardson untuk $p = 1, 2,$ dan 4; tabel multi-titik; grafik hampiran dan galat; serta mini-project yang mengubah data kumulatif penyakit menular menjadi estimasi laju perubahan melalui fitting dan turunan numerik.

## 7.1 Dari beda hingga ke ide Richardson

Pada praktikum sebelumnya, beda maju dan mundur memiliki galat utama $O(h)$, sedangkan beda pusat memiliki galat utama $O(h^2)$. Richardson memanfaatkan informasi orde tersebut untuk menghilangkan suku galat utama.

Misalkan $D^*$ adalah besaran yang sebenarnya ingin dihitung. Pada turunan numerik, $D^* = f'(x)$. Notasi $D(h)$ berarti hasil numerik dari metode yang sama ketika ukuran diskretisasi adalah $h$. Secara umum,

$$D(h) = D^* + Ch^p + O(h^q), \qquad q > p. \tag{7.1}$$

Konstanta $C$ adalah koefisien galat utama. Nilainya berasal dari suku Taylor pertama yang tidak berhasil dieliminasi oleh formula numerik. Contohnya, untuk beda maju $C = f''(x)/2$ dan $p = 1$, sedangkan untuk beda pusat $C = f^{(3)}(x)/6$ dan $p = 2$.

Jika metode yang sama dihitung pada grid yang diperhalus dengan faktor $r > 1$,

$$D(h/r) = D^* + C\left(\frac{h}{r}\right)^p + O(h^q). \tag{7.2}$$

Kalikan Persamaan (7.2) dengan $r^p$, lalu kurangkan Persamaan (7.1). Suku $Ch^p$ saling menghapus, sehingga diperoleh

$$\boxed{D_R = \frac{r^p D(h/r) - D(h)}{r^p - 1}} \tag{7.3}$$

Untuk faktor perhalusan yang paling umum, $r = 2$,

$$D_R = D(h/2) + \frac{D(h/2) - D(h)}{2^p - 1}. \tag{7.4}$$

Catatan numerik. $D(h)$ dan $D(h/2)$ bukan dua metode berbeda. Keduanya berasal dari metode dasar yang sama pada dua resolusi. $D_R$ adalah kombinasi keduanya untuk membatalkan galat utama.

## 7.2 Memilih orde $p$

Nilai $p$ mengikuti orde galat metode dasar. Beberapa contoh:

| Metode dasar | $p$ | $D(h)$ | Richardson, $r = 2$ |
|---|---|---|---|
| Beda maju | 1 | $\dfrac{f(x+h) - f(x)}{h}$ | $D_R = 2D(h/2) - D(h)$ |
| Beda pusat | 2 | $\dfrac{f(x+h) - f(x-h)}{2h}$ | $D_R = \dfrac{4D(h/2) - D(h)}{3}$ |
| Beda pusat orde empat | 4 | $\dfrac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h}$ | $D_R = \dfrac{16D(h/2) - D(h)}{15}$ |

Jika solusi eksak tersedia, orde observasi dapat diperkirakan dari

$$p_{\text{obs}} = \frac{\log(E_h/E_{h/r})}{\log r}, \qquad E_h = |D(h) - D^*|. \tag{7.5}$$

## 7.3 Rumus umum untuk masalah numerik lain

Richardson tidak khusus untuk turunan. Jika suatu kuantitas numerik memenuhi

$$Q(h) = Q^* + Ch^p + \cdots, \tag{7.6}$$

maka pola yang sama dapat dipakai untuk integral numerik, PDB, maupun PDP. Yang berubah hanyalah arti $Q$, parameter diskretisasi $h$, dan orde $p$.

## 7.4 Contoh dari materi kelas

Gunakan fungsi

$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.25, \qquad x_i = 0.5. \tag{7.7}$$

Turunan eksaknya

$$f'(x) = -0.4x^3 - 0.45x^2 - x - 0.25, \qquad f'(0.5) = -0.9125. \tag{7.8}$$

Beda pusat dengan $h = 0.5$ memberi

$$D(0.5) = \frac{f(1) - f(0)}{1} = -1.0000. \tag{7.9}$$

Dengan $h/2 = 0.25$,

$$D(0.25) = \frac{f(0.75) - f(0.25)}{0.5} = -0.934375. \tag{7.10}$$

Karena beda pusat memiliki $p = 2$,

$$D_R = \frac{4D(0.25) - D(0.5)}{3} = -0.9125. \tag{7.11}$$

## 7.5 Dari satu titik ke banyak titik

Pada aplikasi, turunan dihitung pada sekumpulan titik

$$x_1, x_2, \ldots, x_m. \tag{7.12}$$

Untuk setiap $x_i$, hitung $D(h; x_i)$, $D(h/2; x_i)$, dan $D_R(x_i)$, kemudian tampilkan sebagai tabel dan kurva. Untuk data tabel murni, $h/2$ belum tentu tersedia. Karena itu, salah satu strategi adalah melakukan fitting/interpolasi terlebih dahulu, lalu mengevaluasi model halus pada titik yang dibutuhkan.

## 7.6 Program MATLAB: Richardson generik dan multi-titik

numdiff.m

```
function d = numdiff(f,x0,method,h)
%NUMDIFF Turunan pertama dengan beberapa skema beda hingga.
% x0 boleh skalar atau vektor jika f mendukung operasi elementwise.
if nargin < 4 || isempty(h)
    h = eps^(1/3)*max(1,max(abs(x0(:))));
end

switch lower(method)
    case 'forward'
        d = (f(x0+h)-f(x0))/h;
```

```
    case 'backward'
        d = (f(x0)-f(x0-h))/h;
    case 'central'
        d = (f(x0+h)-f(x0-h))/(2*h);
    case {'central4','central-4','fourth'}
        d = (f(x0-2*h)-8*f(x0-h)+8*f(x0+h)-f(x0+2*h))/(12*h);
    otherwise
        error('Metode: forward, backward, central, atau central4.');
end
end
```

### richardson_extrap.m

```
function [DR,Dh,Dhr] = richardson_extrap(approx,h,p,r)
%RICHARDSON_EXTRAP Ekstrapolasi Richardson generik.
% approx(h) mengembalikan hasil metode dasar pada ukuran langkah h.
if nargin < 4 || isempty(r)
    r = 2;
end
if p <= 0 || r <= 1
    error('Gunakan p > 0 dan r > 1.');
end
Dh  = approx(h);
Dhr = approx(h/r);
DR  = (r^p*Dhr-Dh)/(r^p-1);
end
```

### driver_richardson.m

```
%% Praktikum 6: Richardson umum, multi-titik, dan beberapa orde
clear; clc; close all; format long g

f  = @(x) -0.1*x.^4 - 0.15*x.^3 - 0.5*x.^2 - 0.25*x + 1.25;
df = @(x) -0.4*x.^3 - 0.45*x.^2 - x - 0.25;

xq = [0 0.25 0.5 0.75 1.0];
h = 0.25; r = 2;
methods = {'forward','central','central4'};
porder = [1 2 4];
Dexact = df(xq);

figure;
tiledlayout(2,1);
nexttile;
plot(xq,Dexact,'k-','LineWidth',1.8,'DisplayName','Eksak'); hold on

rows = [];
for m = 1:numel(methods)
    approx = @(hh) numdiff(f,xq,methods{m},hh);
    [DR,Dh,Dhr] = richardson_extrap(approx,h,porder(m),r);
    for i = 1:numel(xq)
        rows = [rows; m xq(i) porder(m) Dh(i) Dhr(i) DR(i) ...
            Dexact(i) abs(Dh(i)-Dexact(i)) abs(DR(i)-Dexact(i))]; %#ok<AGROW>
    end
    plot(xq,DR,'o-','DisplayName',['Richardson ' methods{m}]);
end
grid on; xlabel('x'); ylabel('f''(x)');
title('Turunan pada banyak titik'); legend('Location','best');
```

```
T = array2table(rows, 'VariableNames', ...
    {'methodID','x','p','Dh','Dhr','DR','exact','errDh','errDR'});
disp(T)

x0 = 0.5;
hs = 0.4./2.^(0:8);
E = zeros(numel(hs),3); ER = E;
for m = 1:numel(methods)
    for k = 1:numel(hs)
        approx = @(hh) numdiff(f,x0,methods{m},hh);
        [DR,Dh] = richardson_extrap(approx,hs(k),porder(m),r);
        E(k,m)  = abs(Dh-df(x0));
        ER(k,m) = abs(DR-df(x0));
    end
end

nexttile;
loglog(hs,E(:,1),'o-',hs,ER(:,1),'o--', ...
       hs,E(:,2),'s-',hs,ER(:,2),'s--', ...
       hs,E(:,3),'^-',hs,ER(:,3),'^--','LineWidth',1.1);
grid on; xlabel('h'); ylabel('galat absolut');
title('Beda hingga biasa versus Richardson');
legend('Forward','R forward','Central','R central','Central-4','R central-4', ...
       'Location','best');
```

## 7.7 Mini-project: dari data kumulatif penyakit menular ke laju kasus

Misalkan tersedia data sintetik jumlah kasus kumulatif $C(t)$:

| $t$ (hari) | 0 | 2 | 4 | 6 | 8 | 10 | 12 | 14 | 16 | 18 | 20 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $C(t)$ | 6 | 9 | 15 | 26 | 43 | 68 | 98 | 129 | 153 | 170 | 180 |

Data ini hanya untuk latihan numerik dan bukan data surveilans nyata.

Jika $C(t)$ difit dengan kurva halus $\widehat{C}(t)$, maka

$$I(t) = \frac{d\widehat{C}(t)}{dt} \tag{7.13}$$

dapat dibaca sebagai laju perubahan model kasus kumulatif, dalam satuan kasus per hari.

Model awal yang dipilih adalah logistik

$$\widehat{C}(t) = \frac{K}{1 + \exp[-r(t - t_0)]}. \tag{7.14}$$

Parameter ditentukan dengan nonlinear least squares. Dari tabel di atas diperoleh

$$K = 194.6331, \qquad r = 0.3165, \qquad t_0 = 11.9253.$$

Dengan demikian, model yang digunakan pada mini-project adalah

$$\boxed{\widehat{C}(t) = \frac{194.6331}{1 + \exp[-0.3165(t - 11.9253)]}} \tag{7.15}$$

Parameter pada Persamaan (7.15) ditulis empat angka desimal untuk penyajian. Program MATLAB menggunakan nilai hasil optimasi dengan presisi penuh.

Turunan analitik model fit, yang dipakai hanya sebagai verifikasi, adalah

$$\widehat{C}'(t) = \frac{61.6110\,\exp[-0.3165(t-11.9253)]}{\left(1+\exp[-0.3165(t-11.9253)]\right)^2}. \tag{7.16}$$

Sebagai contoh, pada $t = 12$ hari gunakan beda pusat dengan $h = 2$ hari:

$$D(2) = \frac{\widehat{C}(14) - \widehat{C}(10)}{4} = \frac{128.1711 - 68.5463}{4} = 14.9062, \tag{7.17}$$

$$D(1) = \frac{\widehat{C}(13) - \widehat{C}(11)}{2} = \frac{113.7116 - 83.1650}{2} = 15.2733. \tag{7.18}$$

Richardson dengan $p = 2$ memberikan

$$D_R(12) = \frac{4D(1) - D(2)}{3} = 15.3957 \quad \text{kasus/hari}. \tag{7.19}$$

Turunan analitik model fit memberi $\widehat{C}'(12) = 15.4006$ kasus/hari, sehingga galat absolut Richardson sekitar 0.0049 kasus/hari.

`driver_epidemi_turunan.m`

```
%% Mini-project: data kumulatif penyakit menular -> fitting -> turunan
% Data sintetik untuk latihan numerik, bukan data surveilans.
clear; clc; close all; format long g

here = fileparts(mfilename('fullpath'));
T = readtable(fullfile(here,'..','data','data_kasus_kumulatif_sintetik.csv'));
t = T.hari;
C = T.kasus_kumulatif;

% Logistic fit dengan parameter K=exp(a), r=exp(b), t0=c.
theta0 = [log(1.10*max(C)); log(0.30); median(t)];
obj = @(th) sum((exp(th(1))./(1+exp(-exp(th(2))*(t-th(3)))) - C).^2);
opts = optimset('Display','off','TolX',1e-10,'TolFun',1e-10);
theta = fminsearch(obj,theta0,opts);

K  = exp(theta(1));
rr = exp(theta(2));
t0 = theta(3);
Cfit  = @(tt) K./(1+exp(-rr*(tt-t0)));
dCfit = @(tt) K*rr.*exp(-rr*(tt-t0))./(1+exp(-rr*(tt-t0))).^2;

Cpred = Cfit(t);
rmse = sqrt(mean((Cpred-C).^2));

fprintf('K = %.4f, r = %.4f, t0 = %.4f\n',K,rr,t0);
fprintf('Model fit: C(t) = %.4f / (1 + exp(-%.4f*(t-%.4f)))\n',K,rr,t0);
fprintf('RMSE fitting = %.4f kasus\n',rmse);

tq = (2:2:18)';
h = 2; p = 2; refine = 2;
approx = @(hh) numdiff(Cfit,tq,'central',hh);
```

```
[IR,Ih,Ih2] = richardson_extrap(approx,h,p,refine);
Iref = dCfit(tq);

Tout = table(tq,Ih,Ih2,IR,Iref,abs(IR-Iref), ...
    'VariableNames',{'hari','D_h','D_h2','Richardson','turunan_fit','galat_R'});
disp(Tout)

[peakRate,idx] = max(IR);
fprintf('Laju maksimum Richardson = %.4f kasus/hari pada hari %.1f\n', ...
    peakRate,tq(idx));

tfine = linspace(min(t),max(t),400)';
figure;
plot(t,C,'ko','MarkerFaceColor','k','DisplayName','Data sintetik'); hold on
plot(tfine,Cfit(tfine),'LineWidth',1.8,'DisplayName','Fit logistik');
grid on; xlabel('Hari'); ylabel('Kasus kumulatif');
title('Data kumulatif dan model hasil fitting');
legend('Location','southeast');

figure;
plot(tq,Iref,'k-','LineWidth',1.8,'DisplayName','Turunan analitik fit'); hold on
plot(tq,Ih,'o-','DisplayName','Beda pusat D(h)');
plot(tq,Ih2,'s-','DisplayName','Beda pusat D(h/2)');
plot(tq,IR,'d-','LineWidth',1.3,'DisplayName','Richardson');
grid on; xlabel('Hari'); ylabel('Estimasi kasus baru per hari');
title('Laju perubahan kasus kumulatif pada banyak titik');
legend('Location','best');
```

Driver menghasilkan dua visual:

i. data kumulatif dan kurva hasil fitting $\widehat{C}(t)$;

ii. laju perubahan pada banyak titik: $D(h)$, $D(h/2)$, Richardson $D_R$, dan turunan analitik model fit sebagai pembanding verifikasi.

## 7.8 Latihan berorientasi project

i. Beda hingga biasa versus Richardson pada model mini-project. Gunakan Persamaan (7.15) pada $t = 12$ hari. Hitung beda maju untuk $h = 2$ dan $h = 1$, lalu Richardson dengan $p = 1$. Ulangi dengan beda pusat dan Richardson $p = 2$. Bandingkan semua hasil dengan Persamaan (7.16).

ii. Reproduksi contoh manual. Hitung $\widehat{C}(10)$, $\widehat{C}(11)$, $\widehat{C}(13)$, dan $\widehat{C}(14)$ langsung dari Persamaan (7.15). Reproduksi $D(2)$, $D(1)$, dan $D_R(12)$, lalu cocokkan dengan MATLAB.

iii. Turunan pada banyak titik. Untuk $t_i = 4, 6, 8, 10, 12, 14, 16$, hitung $D(2; t_i)$, $D(1; t_i)$, dan $D_R(t_i)$. Tampilkan tabel dan grafik. Tentukan hari ketika laju Richardson maksimum dan jelaskan maknanya.

iv. Eksplorasi orde. Pada $t = 8, 12, 16$, bandingkan Richardson dari beda maju ($p = 1$), beda

pusat ($p = 2$), dan beda pusat orde empat ($p = 4$). Catat jumlah evaluasi fungsi dan galat masing-masing.

v. Pipeline project lengkap. Mulai dari file `data_kasus_kumulatif_sintetik.csv`, lakukan

$$\text{data} \to \text{fitting} \to \widehat{C}(t) \to D(h) \to D(h/2) \to D_R.$$

Laporkan parameter fit, RMSE, grafik fitting, grafik laju perubahan, hari puncak laju, dan galat maksimum Richardson terhadap turunan analitik model fit.

vi. Sensitivitas ukuran langkah. Pada $t = 12$, gunakan $h = 4, 2, 1, 0.5, 0.25$. Plot galat beda pusat dan Richardson terhadap $h$ pada skala log-log. Hitung orde observasi.

vii. Pengembangan menuju project nyata. Ganti data sintetik dengan satu deret waktu penyakit menular yang memiliki data kumulatif atau dapat dikonstruksi menjadi kumulatif. Data nyata harus disertai sumber. Lakukan fitting, estimasi turunan pada banyak titik, Richardson, dan interpretasi periode pertumbuhan tercepat. Jelaskan bahwa turunan model fit adalah laju perubahan model, bukan otomatis jumlah kasus baru terobservasi tanpa galat.

viii. Perbandingan model. Gunakan data yang sama tetapi bandingkan dua model halus, misalnya logistik dan spline. Apakah waktu puncak laju perubahan tetap sama? Jelaskan sensitivitas kesimpulan terhadap model fitting.

Checklist analisis. Laporan yang kuat tidak berhenti pada "Richardson lebih akurat". Jelaskan orde galat, biaya evaluasi fungsi, rentang $h$ yang dipakai, konsistensi di banyak titik, dan arti turunan pada konteks model/data.

## 7.9 Latihan lanjutan dari Lembar Kerja

B5.5 (L5) Jelaskan mengapa pemilihan $h$ yang terlalu besar maupun terlalu kecil dapat menghasilkan kesalahan besar pada turunan numerik. Kaitkan dengan truncation error dan round-off error. Tambahkan satu eksperimen Richardson untuk menunjukkan bahwa peningkatan orde tidak menghilangkan masalah pembulatan ketika $h$ sangat kecil.

Bab 8

# Praktikum 7: Integral Numerik Dasar

Capaian pembelajaran praktikum
Mahasiswa mampu menerapkan aturan Trapezoid dan Simpson serta mengevaluasi galat integrasi numerik (indikator 6.1.8).

Target luaran
Implementasi Trapezoid, Titik Tengah, dan Simpson komposit, studi jejaring perhalusan, serta perbandingan orde galat.

## 8.1 Materi singkat dan landasan teori

Integral tentu dapat dihampiri dengan mengintegralkan interpolan polinomial. Aturan Trapezoid memakai interpolan linear pada setiap subinterval. Untuk jejaring seragam $h = (b-a)/n$,

$$I_T = h\left[\frac{1}{2}f(x_0) + \sum_{i=1}^{n-1} f(x_i) + \frac{1}{2}f(x_n)\right], \tag{8.1}$$

dengan galat global $O(h^2)$ untuk fungsi cukup mulus.

Aturan Titik Tengah juga berorde dua:

$$I_M = h\sum_{i=1}^{n} f\left(a + \left(i - \frac{1}{2}\right)h\right). \tag{8.2}$$

Aturan Simpson 1/3 mengintegralkan interpolan kuadratik dan, untuk $n$ genap,

$$I_S = \frac{h}{3}\left[f_0 + f_n + 4\sum_{i\ ganjil} f_i + 2\sum_{i\ genap,\, i\neq 0,n} f_i\right], \tag{8.3}$$

dengan galat global $O(h^4)$.

## 8.2 Algoritma

i. Pilih $n$, hitung $h = (b-a)/n$ dan titik grid.

ii. Evaluasi $f$ secara vektor bila memungkinkan.

iii. Terapkan bobot sesuai metode.

iv. Ulangi untuk $n, 2n, 4n, \ldots$ dan amati penurunan galat.

v. Pastikan $n$ genap untuk Simpson 1/3 komposit.

## 8.3 Contoh

Untuk $\int_0^\pi \sin x\,dx = 2$, gunakan $n = 4, 8, 16, \ldots$. Trapezoid dan Titik Tengah menunjukkan galat berorde dua, sedangkan Simpson turun jauh lebih cepat karena berorde empat pada fungsi mulus.

## 8.4 Program MATLAB: pisahkan function dan driver

Driver: parameter/data dan grafik. Function: algoritma numerik yang menerima input dan mengembalikan output. Semua file berada pada `Modul_Praktikum/matlab`.

`trap_comp.m (function)`

```
function I=trap_comp(f,a,b,n)
h=(b-a)/n; x=linspace(a,b,n+1); y=f(x);
I=h*(0.5*y(1)+sum(y(2:end-1))+0.5*y(end));
end
```

`midpoint_comp.m (function)`

```
function I=midpoint_comp(f,a,b,n)
h=(b-a)/n; xm=a+h*((1:n)-0.5); I=h*sum(f(xm));
end
```

`simpson_comp.m (function)`

```
function I=simpson_comp(f,a,b,n)
if mod(n,2)~=0, error('Simpson 1/3 komposit memerlukan n genap.'); end
h=(b-a)/n; x=linspace(a,b,n+1); y=f(x);
I=h/3*(y(1)+y(end)+4*sum(y(2:2:end-1))+2*sum(y(3:2:end-2)));
end
```

`driver_integral.m (driver)`

```
%% Praktikum 6: Trapezoid, Midpoint, Simpson
clear; clc; format long g
f=@(x) sin(x); a=0; b=pi; Iexact=2;
ns=[4 8 16 32 64];
for k=1:numel(ns)
    n=ns(k);
    It=trap_comp(f,a,b,n); Im=midpoint_comp(f,a,b,n); Is=simpson_comp(f,a,b,n);
    T(k,:)=[n,It,abs(It-Iexact),Im,abs(Im-Iexact),Is,abs(Is-Iexact)];
end
disp(array2table(T,'VariableNames',{'n','Trap','errT','Mid','errM','Simp','errS'})
    );
```

Catatan numerik. Aturan dengan orde tinggi tidak selalu unggul pada data tidak mulus atau noisy. Asumsi regularitas fungsi harus dipertimbangkan sebelum menyimpulkan metode terbaik.

## 8.5 Luaran driver dan grafik galat

Jalankan driver dan tampilkan keluaran numerik, nilai acuan bila tersedia, serta grafik berlabel lengkap. Plot galat terhadap ukuran langkah $h$ atau banyak subinterval $n$; simpan nilainya sebagai `errs` pada driver bila diperlukan.

## 8.6 Latihan

i. Evaluasi $\int_0^{\pi/2}(6+3\cos x)\,dx$ dengan beberapa $n$ dan hitung galat relatif.

ii. Terapkan Trapezoid pada data tabular $x=[0,0.1,\ldots,0.5]$ dengan $f=[1,8,4,3.5,5,1]$.

iii. Buat grafik galat terhadap $n$ dan estimasikan orde observasi.

iv. Jelaskan kondisi ketika data tabular membuat Simpson tidak dapat langsung diterapkan.

v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.

vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.

vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.

## 8.7 Latihan terintegrasi dari Lembar Kerja

Jika integral analitik tersedia, hitung nilai eksaknya terlebih dahulu dan gunakan sebagai pembanding.

B6.1 (L1) Hitung nilai eksak $I = \int_0^2 (x^2 + 1)\, dx$.

B6.2 (L2) Gunakan aturan Trapezoid satu panel untuk menghampiri integral pada B6.1. Hitung galat absolut dan galat relatif sebenarnya.

B6.3 (L3) Gunakan Simpson 1/3 satu segmen untuk menghampiri integral pada B6.1. Bandingkan hasilnya dengan Trapezoid.

Bab 9

# Praktikum 8: Titik Tengah dan Romberg

Capaian pembelajaran praktikum
Mahasiswa mampu menerapkan Titik Tengah dan Romberg serta membandingkan akurasinya dengan metode integrasi lain; mampu mereproduksi hasil integrasi numerik dari artikel (indikator 6.1.9 dan 7.1.1).

Target luaran
Tabel Romberg, pembandingan Trapezoid/Titik Tengah/Simpson/Romberg, dan analisis biaya terhadap akurasi.

## 9.1 Materi singkat dan landasan teori

Romberg menggabungkan perhalusan berturut-turut Trapezoid dengan Richardson ekstrapolasi. Kolom pertama adalah

$$R_{j,1} = T(h_j), \qquad h_j = \frac{b-a}{2^{j-1}}. \tag{9.1}$$

Kolom selanjutnya dibentuk rekursif

$$R_{j,k} = R_{j,k-1} + \frac{R_{j,k-1} - R_{j-1,k-1}}{4^{k-1} - 1}, \qquad k \le j. \tag{9.2}$$

Karena galat utama Trapezoid mengandung pangkat genap $h^2, h^4, \ldots$, tiap ekstrapolasi menghilangkan satu suku galat utama. Tabel Romberg juga memberikan cara alami memantau konvergensi tanpa solusi analitik: bandingkan elemen diagonal successive.

Titik Tengah berguna sebagai pembanding karena memakai titik interior dan memiliki galat $O(h^2)$ dengan konstanta berbeda dari Trapezoid. Dalam praktik, metode terbaik dipilih berdasarkan kemulusan, biaya evaluasi fungsi, dan target akurasi, bukan orde saja.

## 9.2 Algoritma Romberg

**Algorithm 1** Integrasi Romberg

1: for $j = 1, \ldots, m$ do
2: Hitung $R_{j,1} = T$ dengan $n = 2^{j-1}$ subinterval.
3: for $k = 2, \ldots, j$ do
4: $R_{j,k} \leftarrow R_{j,k-1} + [R_{j,k-1} - R_{j-1,k-1}]/(4^{k-1} - 1)$.
5: end for
6: end for
7: Ambil $R_{m,m}$ atau hentikan ketika diagonal sudah memenuhi toleransi.

## 9.3 Contoh

Hampiri

$$I = \int_0^2 \frac{e^x \sin x}{1 + x^2}\, dx. \tag{9.3}$$

Nilai referensi numerik presisi tinggi sekitar 1.94013002203. Tabel Romberg memperlihatkan bagaimana perhalusan berturut-turut dan ekstrapolasi menghasilkan konvergensi cepat tanpa langsung memanggil penyelesai numerik bawaan sebagai solusi utama.

## 9.4 Program MATLAB: pisahkan function dan driver

Driver: parameter/data dan grafik. Function: algoritma numerik yang menerima input dan mengembalikan output. Semua file berada pada `Modul_Praktikum/matlab`.

`romberg_fun.m (function)`

```
function [R,I]=romberg_fun(f,a,b,m)
R=nan(m,m);
for j=1:m
    n=2^(j-1); R(j,1)=trap_comp(f,a,b,n);
    for k=2:j
        R(j,k)=R(j,k-1)+(R(j,k-1)-R(j-1,k-1))/(4^(k-1)-1);
    end
end
I=R(m,m);
end
```

`trap_comp.m (function)`

```
function I=trap_comp(f,a,b,n)
h=(b-a)/n; x=linspace(a,b,n+1); y=f(x);
I=h*(0.5*y(1)+sum(y(2:end-1))+0.5*y(end));
end
```

`driver_romberg.m (driver)`

```
%% Praktikum 7: Romberg
clear; clc; format long g
```

```
f=@(x) exp(x).*sin(x)./(1+x.^2); a=0; b=2;
[R,I]=romberg_fun(f,a,b,6);
Iref=integral(f,a,b,'AbsTol',1e-13,'RelTol',1e-13);
disp(R); fprintf('Romberg=%.12f, referensi=%.12f, error=%.3e\n',I,Iref,abs(I-Iref)
    );
```

Checklist analisis. Bandingkan bukan hanya galat akhir, tetapi juga jumlah evaluasi $f$. Penyelesai numerik dengan galat kecil tetapi evaluasi sangat banyak belum tentu efisien.

## 9.5 Luaran driver dan grafik galat

Jalankan driver dan tampilkan keluaran numerik, nilai acuan bila tersedia, serta grafik berlabel lengkap. Plot galat terhadap ukuran langkah $h$ atau banyak subinterval $n$; simpan nilainya sebagai `errs` pada driver bila diperlukan.

## 9.6 Latihan

i. Gunakan Romberg untuk $\int_0^3 xe^{2x}\,dx$ dan bandingkan dengan solusi analitik.

ii. Buat kriteria berhenti berdasarkan $|R_{j,j} - R_{j-1,j-1}|$.

iii. Bandingkan Trapezoid, Titik Tengah, Simpson, dan Romberg pada fungsi dengan perubahan tajam.

iv. Reproduksi satu hasil integral numerik dari artikel. Laporkan grid, toleransi, dan penyelesai numerik pembanding.

v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.

vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.

vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.

## 9.7 Latihan lanjutan dari Lembar Kerja

B6.4 (L4) Diberikan tabel $x : 0, 0.5, 1, 1.5, 2$ dan $f(x) : 101, -2, 54, 147, -10$. Hitung hampiran integral $\int_0^2 f(x)\,dx$ dengan Composite Trapezoid dan Composite Simpson 1/3.

B6.5 (L5) Jelaskan kapan hasil integral numerik dapat dievaluasi dengan galat sebenarnya, dan kapan hanya galat hampiran atau perbandingan antarmetode yang dapat dipakai. Berikan masing-masing satu contoh singkat.

Bab 10

# Praktikum 9: Masalah Nilai Awal - Euler dan Runge-Kutta

Capaian pembelajaran praktikum
Mahasiswa mampu menerapkan metode Euler untuk masalah nilai awal dan mengevaluasi galat lokal/global (indikator 6.1.10), serta memperoleh pengantar implementasi Runge-Kutta untuk pembandingan.

Target luaran
Penyelesai numerik Euler dan RK4 modular, kurva solusi, galat terhadap solusi referensi, dan studi ukuran langkah.

## 10.1 Materi singkat dan landasan teori

Masalah nilai awal berbentuk

$$y'(t) = f(t, y), \qquad y(t_0) = y_0. \tag{10.1}$$

Ekspansi Taylor satu langkah memberi

$$y(t + h) = y(t) + hy'(t) + O(h^2), \tag{10.2}$$

sehingga Euler eksplisit

$$y_{n+1} = y_n + hf(t_n, y_n). \tag{10.3}$$

Galat pemotongan lokal per langkah adalah $O(h^2)$, sedangkan galat global setelah banyak langkah adalah $O(h)$. Ini menjelaskan mengapa pembagian ukuran langkah menjadi dua idealnya mengurangi galat global kira-kira faktor dua pada rentang asimtotik.

### Metode Heun (prediktor–korektor)

Dua evaluasi kemiringan meningkatkan akurasi terhadap Euler. Prediktor dan korektor adalah

$$\widetilde{y}_{n+1} = y_n + hf(t_n, y_n), \tag{10.4}$$

$$y_{n+1} = y_n + \frac{h}{2}\big[f(t_n, y_n) + f(t_n + h, \widetilde{y}_{n+1})\big]. \tag{10.5}$$

Metode Heun inilah yang digunakan pada B7.2 dan Mock-Up UAS.3, sehingga tidak cukup hanya menampilkan Euler dan RK4.

Runge-Kutta orde empat (RK4) mencapai akurasi global $O(h^4)$ tanpa membutuhkan turunan eksplisit $f$. Ia menggabungkan empat estimasi kemiringan:

$$k_1 = f(t_n, y_n), \tag{10.6}$$

$$k_2 = f(t_n + h/2, y_n + hk_1/2), \tag{10.7}$$

$$k_3 = f(t_n + h/2, y_n + hk_2/2), \tag{10.8}$$

$$k_4 = f(t_n + h, y_n + hk_3), \tag{10.9}$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{10.10}$$

## 10.2 Contoh model

Model logistik

$$y' = ry\left(1 - \frac{y}{K}\right), \qquad y(0) = y_0 \tag{10.11}$$

memiliki solusi analitik

$$y(t) = \frac{K}{1 + (K/y_0 - 1)e^{-rt}}. \tag{10.12}$$

Dengan $K = 100$, $r = 0.4$, dan $y_0 = 10$, nilai pada $t = 10$ sekitar 85.8486. Solusi analitik menjadi verifikasi case untuk menguji implementasi Euler/RK4.

## 10.3 Algoritma

i. Bentuk grid $t_n = t_0 + nh$.

ii. Tetapkan $y_0$.

iii. Untuk tiap langkah, evaluasi formula Euler atau RK4.

iv. Simpan seluruh pasangan $(t_n, y_n)$.

v. Jika referensi tersedia, hitung galat maksimum dan ulangi untuk beberapa $h$.

## 10.4 Program MATLAB: pisahkan function dan driver

Driver: parameter/data dan grafik. Function: algoritma numerik yang menerima input dan mengembalikan output. Semua file berada pada `Modul_Praktikum/matlab`.

`euler_ivp.m (function)`

```
function [t,y]=euler_ivp(f,tspan,y0,h)
t=(tspan(1):h:tspan(2))'; if t(end)<tspan(2), t=[t;tspan(2)]; end
y=zeros(numel(t),numel(y0)); y(1,:)=y0(:)';
for n=1:numel(t)-1
    hn=t(n+1)-t(n); y(n+1,:)=y(n,:)+hn*f(t(n),y(n,:))';
end
end
```

`heun_ivp.m (function)`

```
function [t,y] = heun_ivp(f,tspan,y0,h)
%HEUN_IVP Explicit trapezoidal predictor-corrector for scalar/vector ODE.
if h<=0, error('h harus positif');end
t=(tspan(1):h:tspan(2))';
if t(end)<tspan(2),t=[t;tspan(2)];end
n=numel(y0);y=zeros(numel(t),n);y(1,:)=y0(:)';
for k=1:numel(t)-1
    hk=t(k+1)-t(k);tk=t(k);yn=y(k,:)';
    k1=f(tk,yn);predictor=yn+hk*k1;
    k2=f(tk+hk,predictor);
    y(k+1,:)=(yn+hk*(k1+k2)/2)';
end
end
```

`rk4_ivp.m (function)`

```
function [t,y]=rk4_ivp(f,tspan,y0,h)
t=(tspan(1):h:tspan(2))'; if t(end)<tspan(2), t=[t;tspan(2)]; end
y=zeros(numel(t),numel(y0)); y(1,:)=y0(:)';
for n=1:numel(t)-1
    hn=t(n+1)-t(n); yn=y(n,:)'; tn=t(n);
    k1=f(tn,yn); k2=f(tn+hn/2,yn+hn*k1/2);
    k3=f(tn+hn/2,yn+hn*k2/2); k4=f(tn+hn,yn+hn*k3);
    y(n+1,:)=(yn+hn*(k1+2*k2+2*k3+k4)/6)';
end
end
```

`driver_ode.m (driver)`

```
%% DRIVER PDB: Euler, Heun, RK4 versus solusi eksak
clear;clc;close all;format long g
K=100;r=0.4;y0=10;tspan=[0 10];h=0.5;
f=@(t,y) r*y*(1-y/K);
exact=@(t) K./(1+(K/y0-1).*exp(-r*t));
[t,yEuler]=euler_ivp(f,tspan,y0,h);
[~,yHeun]=heun_ivp(f,tspan,y0,h);
[~,yRK4]=rk4_ivp(f,tspan,y0,h);
yTrue=exact(t);
```

```
errs=[max(abs(yEuler-yTrue)),max(abs(yHeun-yTrue)),...
      max(abs(yRK4-yTrue))];
fprintf('Error maksimum Euler/Heun/RK4: %g %g %g\n',errs)
figure;plot(t,yTrue,'k-',t,yEuler,'o-',t,yHeun,'s-',t,yRK4,'^-')
grid on;xlabel('t');ylabel('y(t)');legend('Eksak','Euler','Heun','RK4')
figure;semilogy(t,max(abs(yEuler-yTrue),eps),'o-',...
 t,max(abs(yHeun-yTrue),eps),'s-',...
 t,max(abs(yRK4-yTrue),eps),'^-');grid on
xlabel('t');ylabel('Galat absolut')
legend('Euler','Heun','RK4');title('Perbandingan galat solusi PDB')
```

## 10.5 Luaran driver dan grafik galat

Jalankan driver dan tampilkan keluaran numerik, nilai acuan bila tersedia, serta grafik berlabel lengkap. Bila metode mengulang dalam waktu, plot galat terhadap waktu dan ulangi eksperimen dengan beberapa $h$.

## 10.6 Latihan

i. Uji $h = 1, 0.5, 0.25, 0.125$ pada model logistik dan estimasikan orde global Euler serta RK4.

ii. Terapkan Euler pada $y' = -2y + t$, $y(0) = 1$. Bandingkan dengan solusi analitik.

iii. Modifikasi penyelesai numerik agar dapat menangani sistem ODE dua variabel.

iv. Jelaskan perbedaan galat pemotongan lokal dan galat global dengan kalimat Anda sendiri.

v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.

vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.

vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.

## 10.7 Latihan terintegrasi dari Lembar Kerja

Gunakan solusi eksak ketika tersedia untuk mengevaluasi galat metode.

B7.1 (L1) Diberikan masalah nilai awal $\frac{dy}{dt} = 2y^2(t-1)$, $y(0) = 1$. Gunakan metode Euler dengan $h = 0.2$ untuk menghitung $y_1$.

B7.2 (L2) Untuk masalah pada B7.1, gunakan metode Heun dengan $h = 0.2$ untuk menghitung $y_1$.

Bab 11

# Praktikum 10: Stabilitas, Ukuran Langkah, dan Reproduksi Model PDB

Capaian pembelajaran praktikum
Mahasiswa mampu menerapkan Runge-Kutta dan menganalisis akurasi, stabilitas, serta pengaruh ukuran langkah; mampu mereproduksi kurva/model sederhana dari artikel (indikator 6.1.11 dan 7.1.1).

Target luaran
Studi stabilitas ukuran langkah pada persamaan uji, perbandingan Euler-RK4, serta satu reproduksi kurva model dari literatur.

## 11.1 Materi singkat dan landasan teori

Akurasi tidak cukup: metode ODE juga harus stabil. Persamaan uji

$$y' = \lambda y, \qquad \Re(\lambda) < 0 \tag{11.1}$$

memiliki solusi yang meluruh. Euler memberikan

$$y_{n+1} = (1 + h\lambda) y_n. \tag{11.2}$$

Agar solusi numerik juga meluruh, diperlukan

$$|1 + h\lambda| < 1. \tag{11.3}$$

Untuk $\lambda$ real negatif, syarat ini menjadi $0 < h < -2/\lambda$. Jadi ukuran langkah yang terlalu besar dapat menghasilkan osilasi/growth numerik walaupun solusi kontinu stabil.

RK4 memiliki polinom stabilitas

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \qquad z = h\lambda. \tag{11.4}$$

Region $|R(z)| < 1$ lebih luas daripada Euler, tetapi tetap terbatas. Pada masalah kaku, metode eksplisit dapat memerlukan step sangat kecil demi stabilitas, bukan demi akurasi.

Reproduksi artikel harus membedakan data/model parameter dari parameter numerik. Dua simulasi dengan model dan parameter fisik sama dapat berbeda jika penyelesai numerik, toleransi, atau ukuran langkah tidak sama.

## 11.2 Contoh

Ambil $y' = -15y$, $y(0) = 1$. Euler dengan $h = 0.2$ memiliki faktor amplifikasi $1 - 3 = -2$, sehingga tidak stabil. Dengan $h = 0.1$, faktor $-0.5$ dan iterasi stabil tetapi berosilasi tanda. Ini merupakan contoh sederhana bahwa ukuran langkah adalah bagian dari kualitas solusi.

## 11.3 Program MATLAB: pisahkan function dan driver

Driver: parameter/data dan grafik. Function: algoritma numerik yang menerima input dan mengembalikan output. Semua file berada pada `Modul_Praktikum/matlab`.

`euler_ivp.m (function)`

```
function [t,y]=euler_ivp(f,tspan,y0,h)
t=(tspan(1):h:tspan(2))'; if t(end)<tspan(2), t=[t;tspan(2)]; end
y=zeros(numel(t),numel(y0)); y(1,:)=y0(:)';
for n=1:numel(t)-1
    hn=t(n+1)-t(n); y(n+1,:)=y(n,:)+hn*f(t(n),y(n,:))';
end
end
```

`rk4_ivp.m (function)`

```
function [t,y]=rk4_ivp(f,tspan,y0,h)
t=(tspan(1):h:tspan(2))'; if t(end)<tspan(2), t=[t;tspan(2)]; end
y=zeros(numel(t),numel(y0)); y(1,:)=y0(:)';
for n=1:numel(t)-1
    hn=t(n+1)-t(n); yn=y(n,:)'; tn=t(n);
    k1=f(tn,yn); k2=f(tn+hn/2,yn+hn*k1/2);
    k3=f(tn+hn/2,yn+hn*k2/2); k4=f(tn+hn,yn+hn*k3);
    y(n+1,:)=(yn+hn*(k1+2*k2+2*k3+k4)/6)';
end
end
```

`driver_stabilitas.m (driver)`

```
%% Praktikum 9: stabilitas dan pengaruh step size
clear; clc; format long g
lambda=-15; f=@(t,y) lambda*y; y0=1; tspan=[0 1];
hs=[0.2 0.1 0.05 0.025];
for k=1:numel(hs)
    h=hs(k); [tE,yE]=euler_ivp(f,tspan,y0,h); [tR,yR]=rk4_ivp(f,tspan,y0,h);
```

```
    exact=exp(lambda*tE);
    T(k,:)=[h,max(abs(yE-exact)),max(abs(yR-exact))];
end
disp(array2table(T,'VariableNames',{'h','errEuler','errRK4'}));
% Euler stabil untuk masalah uji y'=lambda*y jika |1+h*lambda|<1.
```

Checklist analisis. Pada reproduksi artikel, catat: model, parameter, kondisi awal, interval waktu, penyelesai numerik, ukuran langkah/tolerance, dan pengolahan akhir. Jika salah satu tidak tersedia, nyatakan asumsi secara eksplisit.

## 11.4 Luaran driver dan grafik galat

Jalankan driver dan tampilkan keluaran numerik, nilai acuan bila tersedia, serta grafik berlabel lengkap. Bila metode mengulang dalam waktu, plot galat terhadap waktu dan ulangi eksperimen dengan beberapa $h$.

## 11.5 Latihan

i. Untuk $\lambda = -5, -15, -50$, tentukan batas stabilitas Euler secara analitik lalu uji dengan kode.

ii. Buat grafik fungsi stabilitas $|1 + z|$ sepanjang sumbu real negatif dan tandai interval stabil Euler.

iii. Reproduksi satu kurva ODE dari artikel/model yang Anda pilih. Lakukan jejaring perhalusan minimal dua kali.

iv. Bandingkan hasil RK4 manual dengan `ode45` sebagai penyelesai numerik pembanding, bukan sebagai pengganti analisis.

v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.

vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.

vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.

## 11.6 Latihan lanjutan dari Lembar Kerja

B7.3 (L3) Gunakan RK4 klasik dengan $h = 0.2$ untuk menghitung $y_1$ pada masalah yang sama. Tuliskan $k_1, k_2, k_3, k_4$.

B7.4 (L4) Diketahui solusi eksak masalah pada B7.1 adalah $y(t) = \frac{1}{1+2t-t^2}$. Hitung nilai eksak $y(0.2)$, lalu evaluasi galat absolut ketiga metode pada B7.1–B7.3.

B7.5 (L5) Bandingkan Euler, Heun, dan RK4 dari aspek biaya komputasi per langkah, akurasi, dan kecocokan untuk masalah dengan dinamika halus.

Bab 12

# Praktikum 11: Praktikum Terpadu, Verifikasi, dan Validasi

Capaian pembelajaran praktikum
Mahasiswa mampu memilih metode numerik yang sesuai, mengintegrasikan beberapa metode dalam satu alur kerja, memeriksa keakuratan dan keterbatasan hasil, serta mendokumentasikan proses komputasi secara ilmiah (penguatan CPMK 6.1 dan 7.1).

Target luaran
Satu mini-pipeline komputasi yang menghubungkan formulasi masalah, penyelesai numerik, verifikasi, analisis sensitivitas numerik, dan interpretasi luaran sebagai persiapan UAS dan proyek.

## 12.1 Materi singkat: dari metode ke alur kerja

Pada masalah nyata, metode numerik jarang berdiri sendiri. Sebuah model dapat memerlukan pencarian akar untuk event time, penyelesaian SPL di dalam iterasi nonlinear, turunan/integral untuk kuantitas turunan, dan penyelesai numerik ODE untuk dinamika. Karena itu mahasiswa perlu beralih dari pertanyaan "rumus apa?" menjadi "alur kerja apa yang dapat dipertanggungjawabkan?".

Alur kerja yang disarankan:

i. Formulasi: definisikan variabel, parameter, persamaan, domain, kondisi awal/batas, dan kuantitas yang ingin dihitung.

ii. Pemilihan metode: cocokkan struktur masalah dengan penyelesai numerik dan pahami asumsi metode.

iii. Implementasi: gunakan fungsi modular, parameter numerik eksplisit, dan penyimpanan history.

iv. Verifikasi: gunakan solusi analitik, masalah uji terancang, residual, pemeriksaan konservasi, atau jejaring perhalusan.

v. Validasi: bila data tersedia, bandingkan model dengan data/fenomena menggunakan metrik yang relevan.

vi. Sensitivitas numerik: ubah $h$, toleransi, tebakan awal, dan penyelesai numerik untuk memastikan kesimpulan tidak merupakan artefak numerik.

vii. Pelaporan: simpan tabel, grafik, kode, konfigurasi, dan interpretasi.

## 12.2 Contoh terpadu: model logistik

Gunakan model logistik pada Praktikum 8. Pertanyaan komputasi dapat dirangkai sebagai berikut.

i. Kapan $y(t)$ pertama kali mencapai 80? Ini menjadi masalah akar $q(t) = y(t) - 80 = 0$ dan dapat diselesaikan dengan Biseksi. Nilai referensinya sekitar $t = 8.9588$.

ii. Bagaimana kurva dinamikanya? Gunakan RK4 dan bandingkan dengan solusi analitik.

iii. Berapa laju perubahan di $t = 5$? Gunakan turunan numerik sebagai pemeriksaan silang terhadap ruas kanan ODE.

iv. Berapa $\int_0^{10} y(t)\,dt$? Gunakan Simpson sebagai ukuran akumulasi; nilai referensi sekitar 462.50.

v. Apakah hasil stabil jika $h$ RK4 dibagi dua dan toleransi akar diperketat?

## 12.3 Algoritma alur kerja

**Algorithm 2** Mini-pipeline numerik

1: Definisikan model dan parameter.
2: Tentukan masalah uji/solusi acuan untuk verifikasi bila tersedia.
3: Selesaikan event/root quantity.
4: Simulasikan dinamika dengan minimal satu penyelesai numerik utama.
5: Hitung derived quantities (turunan/integral) dengan metode yang sesuai.
6: Lakukan perhalusan/sensitivity numerik.
7: Kumpulkan residual, galat, grafik, dan tabel.
8: Interpretasikan hasil dan nyatakan keterbatasan.

## 12.4 Program MATLAB: pisahkan function dan driver

Driver: parameter/data dan grafik. Function: algoritma numerik yang menerima input dan mengembalikan output. Semua file berada pada `Modul_Praktikum/matlab`.

`bisection_fun.m (function)`

```
function [x,history] = bisection_fun(f,a,b,tol,maxit)
fa=f(a); fb=f(b);
if fa*fb>0, error('Interval awal tidak mengurung akar.'); end
xold = NaN; iter=[]; xs=[]; errs=[]; res=[];
for k=1:maxit
    x=(a+b)/2; fx=f(x);
    if k==1, err=Inf; else, err=abs(x-xold)/max(1,abs(x)); end
    iter(end+1,1)=k; xs(end+1,1)=x; errs(end+1,1)=err; res(end+1,1)=abs(fx);
    if abs(fx)<=tol || err<=tol, break; end
    if fa*fx<0
        b=x; fb=fx;
    else
        a=x; fa=fx;
    end
    xold=x;
end
history=table(iter,xs,errs,res,'VariableNames',{'iter','x','err','residual'});
end
```

`rk4_ivp.m (function)`

```
function [t,y]=rk4_ivp(f,tspan,y0,h)
t=(tspan(1):h:tspan(2))'; if t(end)<tspan(2), t=[t;tspan(2)]; end
y=zeros(numel(t),numel(y0)); y(1,:)=y0(:)';
for n=1:numel(t)-1
    hn=t(n+1)-t(n); yn=y(n,:)'; tn=t(n);
    k1=f(tn,yn); k2=f(tn+hn/2,yn+hn*k1/2);
    k3=f(tn+hn/2,yn+hn*k2/2); k4=f(tn+hn,yn+hn*k3);
    y(n+1,:)=(yn+hn*(k1+2*k2+2*k3+k4)/6)';
end
end
```

`simpson_comp.m (function)`

```
function I=simpson_comp(f,a,b,n)
if mod(n,2)~=0, error('Simpson 1/3 komposit memerlukan n genap.'); end
h=(b-a)/n; x=linspace(a,b,n+1); y=f(x);
I=h/3*(y(1)+y(end)+4*sum(y(2:2:end-1))+2*sum(y(3:2:end-2)));
end
```

`driver_terpadu.m (driver)`

```
%% Praktikum 10: pipeline terintegrasi dan verification & validation
clear; clc; format long g
K=100; r=0.4; y0=10;
yexact=@(t) K./(1+(K/y0-1).*exp(-r*t));
% 1) Pencarian akar: kapan populasi mencapai 80?
q=@(t) yexact(t)-80;
```

```
[t80,hroot]=bisection_fun(q,0,20,1e-10,100);
% 2) Simulasi model dengan RK4
f=@(t,y) r*y*(1-y/K); [t,y]=rk4_ivp(f,[0 10],y0,0.25);
% 3) Turunan numerik pada t=5 dari kurva analitik sebagai verification case
dy5=numdiff(yexact,5,'central',1e-3);
% 4) Integral numerik: area di bawah kurva 0<=t<=10
I=simpson_comp(yexact,0,10,100);
% 5) Residual verifikasi solver terhadap solusi analitik
err=max(abs(y-yexact(t)));
fprintf('t saat y=80       : %.8f\n',t80);
fprintf('dy/dt pada t=5    : %.8f\n',dy5);
fprintf('Integral y dt     : %.8f\n',I);
fprintf('Max error RK4     : %.3e\n',err);
```

## 12.5 Latihan terpadu

i. Buat tabel yang memetakan tipe masalah ke metode yang tersedia pada Praktikum 1–9. Tambahkan kelebihan, keterbatasan, dan parameter numerik utama.

ii. Pilih satu model sederhana (peluruhan, logistik, pendinginan Newton, RC circuit, atau model lain). Bangun alur kerja yang memuat minimal tiga metode numerik berbeda.

iii. Lakukan verifikasi dengan solusi analitik atau manufactured solution. Jika tidak ada, gunakan jejaring/tolerance perhalusan dan jelaskan keterbatasannya.

iv. Buat satu halaman catatan keterulangan: versi MATLAB, nama file, parameter, seed (jika ada bilangan acak), dan langkah menjalankan program.

# Mock-Up UTS dari Lembar Kerja

Cakupan: pencarian akar dan Sistem Persamaan Linear. Butir berikut dipertahankan sama dengan Lembar Kerja agar dapat dipakai untuk simulasi ujian.

## Soal Mock-Up UTS

UTS.1 (L3) Untuk $f(x) = x^3 - x - 1$, gunakan Metode Biseksi pada $[1, 1.5]$ sampai $\varepsilon_a < 2\%$ atau maksimal 6 iterasi. Sajikan hasilnya dalam tabel iterasi.

UTS.2 (L4) Untuk $f(x) = e^{-x} - x$, gunakan Newton-Raphson dari $x_0 = 0$. Tentukan turunannya terlebih dahulu, lakukan 5 iterasi, dan hitung $\varepsilon_a$.

UTS.3 (L4) Susun model tiga reaktor $100c_1 - 20c_3 = 160$, $-40c_1 + 60c_2 = 20$, $-60c_1 - 60c_2 + 120c_3 = 0$ ke bentuk $A\mathbf{c} = \mathbf{b}$, lalu tentukan solusi eksaknya.

UTS.4 (L5) Gunakan Gauss-Seidel untuk model reaktor pada UTS.3 dengan tebakan awal $(0, 0, 0)^T$. Lakukan 3 iterasi dan hitung galat absolut terhadap solusi eksak.

# Mock-Up UAS dari Lembar Kerja

Cakupan: turunan numerik, integral numerik, dan Persamaan Diferensial Biasa. Gunakan solusi analitik/eksak sebagai pembanding bila tersedia.

## Soal Mock-Up UAS

UAS.1 (L3) Untuk $f(x) = x\sqrt{x} - 7x$: (a) tentukan turunan analitik $f'(x)$; (b) hampiri $f'(1.5)$ dengan beda pusat $h = 0.5$; (c) hitung galat relatif sebenarnya.

UAS.2 (L4) Hitung $I = \int_0^2 (x^2 + 1)\, dx$ dengan (a) nilai eksak, (b) Trapezoid satu panel, dan (c) Simpson 1/3. Bandingkan galat kedua hampiran numerik.

UAS.3 (L4) Gunakan Heun untuk masalah $\frac{dy}{dt} = 2y^2(t-1)$, $y(0) = 1$ dengan $h = 0.2$ untuk memperoleh $y_1$. Tuliskan langkah prediktor dan korektornya.

UAS.4 (L5) Untuk masalah pada UAS.3, bandingkan hasil Euler, Heun, dan RK4 pada $t = 0.2$ dengan solusi eksak $y(t) = \frac{1}{1+2t-t^2}$. Berikan simpulan mengenai hubungan biaya komputasi dan akurasi.

# Rubrik Analisis Umum

| Aspek | Pertanyaan yang harus dapat dijawab |
|---|---|
| Formulasi | Apa persamaan, variabel, domain, asumsi, dan kuantitas yang dicari? |
| Metode | Mengapa metode ini sesuai? Apa syarat/risiko kegagalannya? |
| Implementasi | Apakah program modular, dapat dijalankan ulang, dan tidak memakai operasi numerik yang buruk seperti invers eksplisit tanpa alasan? |
| Konvergensi | Apa kriteria berhenti? Bagaimana residual/galat berubah? |
| Akurasi | Apa solusi pembanding? Bagaimana orde/ukuran langkah memengaruhi galat? |
| Ketahanan | Apa yang terjadi jika tebakan awal, toleransi, atau ukuran langkah diubah? |
| Interpretasi | Apa arti angka/kurva dalam konteks model? |
| Keterulangan | Dapatkah orang lain menjalankan kode dan memperoleh hasil yang sama? |

# Daftar File MATLAB

File utama edisi ini:

- Akar dan SPL: `driver_akar.m`, `driver_spl.m`.
- SPNL dan regresi: `driver_spnl.m`, `driver_regresi.m`, `driver_tugas_curvefit.m`.
- Turunan dan integral: `driver_turunan.m`, `driver_richardson.m`, `driver_integral.m`, `driver_romberg.m`.
- PDB dan sintesis: `driver_ode.m`, `driver_stabilitas.m`, `driver_terpadu.m`.

Function yang dipakai dijelaskan di setiap bab. Fungsi tambahan antara lain `relative_error.m`, `regresi_linear.m`, `regresi_nonlinear.m`, serta `metrik_regresi.m`. Nama file lama masih disimpan sebagai kompatibilitas; jalankan driver baru sesuai bab.

Paket kode menyertakan skrip utama dan fungsi berikut.

- `relerr.m`
- `akar_demo.m`
- `spl_demo.m`
- `spnl_demo.m`
- `curvefit_demo.m`
- `root_compare.m`
- `bisection_fun.m`
- `regula_falsi_fun.m`
- `newton_fun.m`
- `secant_fun.m`
- `fixedpoint_fun.m`
- `spl_iterative.m`
- `jacobi_iter.m`
- `gauss_seidel_iter.m`
- `spnl_curvefit.m`
- `newton_system.m`
- `gauss_newton_damped.m`
- `driver_interpolasi.m`
- `interp_poly_spl.m`
- `newton_dd.m`
- `finite_difference.m`
- `numdiff.m`

- `richardson_diff.m`
- `integration_basic.m`
- `trap_comp.m`
- `midpoint_comp.m`
- `simpson_comp.m`
- `romberg_demo.m`
- `romberg_fun.m`
- `ode_euler_rk4.m`
- `euler_ivp.m`
- `rk4_ivp.m`
- `ode_stability.m`
- `integrated_review.m`

# Referensi Utama

i. Chapra, S. C. dan Canale, R. P. (2015). Numerical Methods for Engineers, 7th ed. McGraw-Hill Education.